



第四章 语法制导翻译生成中间代码

语法制导翻译是处理语义的基本方法。

- 扩充描述语言语法结构的CFG，为文法的每个产生式都**附加**一个语义子程序。例如， $E \rightarrow E + T$ {语义子程序}
- 以语法分析为基础，在语法分析过程中，每当需要使用一产生式进行推导或归约时，**语法分析程序**除执行相应的**语法分析动作**外还要执行相应的**语义动作**或调用相应的**语义子程序**，如查填符号表、计算表达式的值、生成中间代码等。

主要包括：

1. 语法制导翻译的基本概念
2. 中间代码简介
3. 典型声明语句的翻译
4. 可执行语句的翻译



4.1 语法制导翻译简介

4.1.1 语法与语义

1. 语法与语义的关系

语法是指语言的结构、即语言的“样子”；语义是指附着于语言结构上的实际含义，即语言的“意义”。

- ① 语义不能离开语法独立存在；
- ② 语义远比语法复杂；
- ③ 同一语言结构可包含多种含义，不同语言结构可表示相同含义；
- ④ 程序设计语言的语法和语义之间没有明确界线，语义可以完成CFG无法描述的特性。例如，标识符的先声明后引用。

[例1] 语法正确的句子语义不一定正确：

“猫吃老鼠”与“老鼠吃猫”



4.1 语法制导翻译简介

2. 语义分析的双重作用

- ① 检查是否结构正确的句子所表示的意思也**合法**，是否存在静态语义错误；
- ② 执行规定的语义动作，如：
 - ✱ 表达式求值
 - ✱ 符号表填写
 - ✱ 中间代码生成
 - ✱

3. 语义分析的方法

语法制导翻译



4.1 语法制导翻译简介

4.1.2 属性与语义规则

1. 语法制导翻译的基本思想

以语法分析为基础，**伴随**语法分析的各个步骤，执行相应的语义动作，达到对语义的处理。

具体方法：

语言结构-----文法符号

语言结构的语义---文法符号的属性

- ① 将文法符号所代表的语言结构的意思，用附着于该文法符号的**属性**表示；
- ② 用**语义规则**规定**产生式**所代表的**语言结构之间的关系**（即属性之间的关系），即用语义规则实现**属性计算**。

语义规则的执行：在语法分析的适当时刻(如推导或归约)，执行附着在对应产生式上的语义规则，以实现^{对语言结构语义}的处理，如计算属性值、查填符号表、生成中间代码、发布出错信息等。



4.1 语法制导翻译简介

2. 属性(attribute)的抽象表示

.attr

如: E.val (值), E.type (类型), E.code (代码序列),
E.place (存储空间)

3. 对文法的约定

只关注语法分析基础上的语义处理, 忽略语法分析。为了简单, 讨论的文法一般为二义文法。默认解决二义的方法是规定常规意义下的优先级和结合性。

4. 属性的定义

定义4.1 对于产生式 $A \rightarrow \alpha$, 其中 α 是由文法符号 $X_1 X_2 \dots X_n$ 组成的序列, 它的**语义规则**可以表示为(4.1)所示关于**属性的函数**:

$$b := f(c_1, c_2, \dots, c_k) \quad (4.1)$$



4.1 语法制导翻译简介

$$A \rightarrow \alpha \qquad b := f(c_1, c_2, \dots, c_k) \qquad (4.1)$$

语义规则中的属性存在下述性质与关系。

- ① 若 b 是 A 的属性， $c_1, c_2, \dots, c_k$ 是 α 中文法符号的属性或者 A 的其他属性，则称 b 是 A 的**综合属性**。
- ② 若 b 是 α 中某文法符号 X_i 的属性， $c_1, c_2, \dots, c_k$ 是 A 的属性，或者是 α 中其它文法符号的属性，则称 b 是 X_i 的**继承属性**。
- ③ 称(4.1)中属性 b **依赖于**属性 $c_1, c_2, \dots, c_k$ 。
- ④ 若语义规则的形式如下述(4.2)，则可将其想像为产生式左部文法符号 A 的一个**虚拟属性**。属性之间的依赖关系，在虚拟属性上依然存在。

$$f(c_1, c_2, \dots, c_k) \quad (4.2)$$

(4.1)中属性之间的依赖关系，实质上**反映了属性计算的先后次序**，即所有属性 c_i 被计算之后才能计算属性 b 。



属性文法

[例4.3] 计算表达式值的语法制导翻译。以 $3+5*8$ 为例。

产生式	语法制导定义
$L \rightarrow E$	{print(E.val);}
$E \rightarrow E_1 + E_2$	{E.val:=E1.val+E2.val;}
$E \rightarrow E_1 * E_2$	{E.val:=E1.val*E2.val;}
$E \rightarrow n$	{E.val:=n.lexval;}

- 在上下文无关文法的基础上为每个文法符号配备若干个相关的“值”（属性）。通过“属性”来刻画文法符号的语义。
- 属性的设置与语法制导翻译的功能有关，实现不同功能的语法制导翻译需要用到的属性可能不同。



4.1 语法制导翻译简介

4.1.3 语义规则的两两种形式

1. 语法制导定义

用**抽象**的属性和运算符表示的语义规则(公式，做什么)

2. 翻译方案

用**具体**的数据结构和运算表示的语义规则（程序段，如何做）

语义规则也被习惯上称为**语义动作**。忽略实现细节，二者作用等价。语法制导定义适用于**设计阶段**，翻译方案适用于**实现阶段**。



4.1 语法制导翻译简介

[例4.1] 将中缀形式的算术表达式转换为后缀表示。其语法制导定义和翻译方案可分别表示如下。其中 $\text{print}(\text{E.post})$ 是L的虚拟属性，可以想象为 $\text{L.p} := \text{print}(\text{E.post})$ 。翻译方案中的 lexval 表示词法分析返回的记号num的值。

产生式	语法制导定义	翻译方案1	翻译方案2
$L \rightarrow E$	$\text{print}(\text{E.post})$	$\text{print_post}(\text{post});$	
$E \rightarrow E1 + E2$	$\text{E.post} := \text{E1.post} \parallel \text{E2.post} \parallel '+';$	$\text{post}(k) := '+';$ $k := k+1;$	$\text{print}(+);$
$E \rightarrow \text{num}$	$\text{E.post} := \text{num.lexval};$	$\text{post}(k) := \text{lexval};$ $k := k+1;$	$\text{print}(\text{lexval});$

语法制导定义：算法

翻译方案：程序实现，方法不唯一



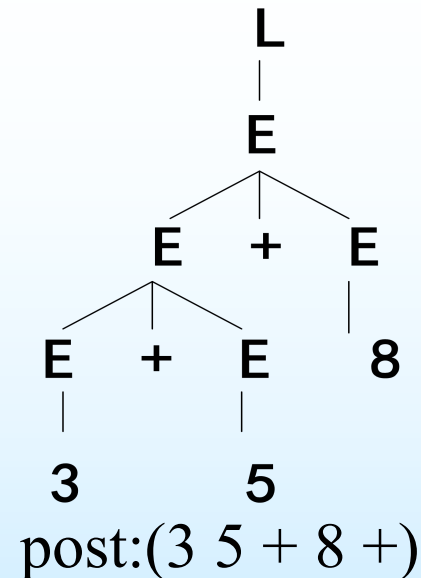
4.1 语法制导翻译简介

翻译方案中需要考虑的问题：

- ① 采用什么样的语法分析方法，不同的分析方法对语义处理的次序不同；
- ② 为属性分配存储空间；
- ③ 考虑计算次序。

翻译方案1，自下而上计算，LR分析。以 $3+5+8$ 为例，归约时翻译。

产生式	翻译方案1
$L \rightarrow E$	<code>print_post(post);</code>
$E \rightarrow E_1 + E_2$	<code>post(k) := '+'; k := k+1;</code>
$E \rightarrow \text{num}$	<code>post(k) := lexval; k := k+1;</code>



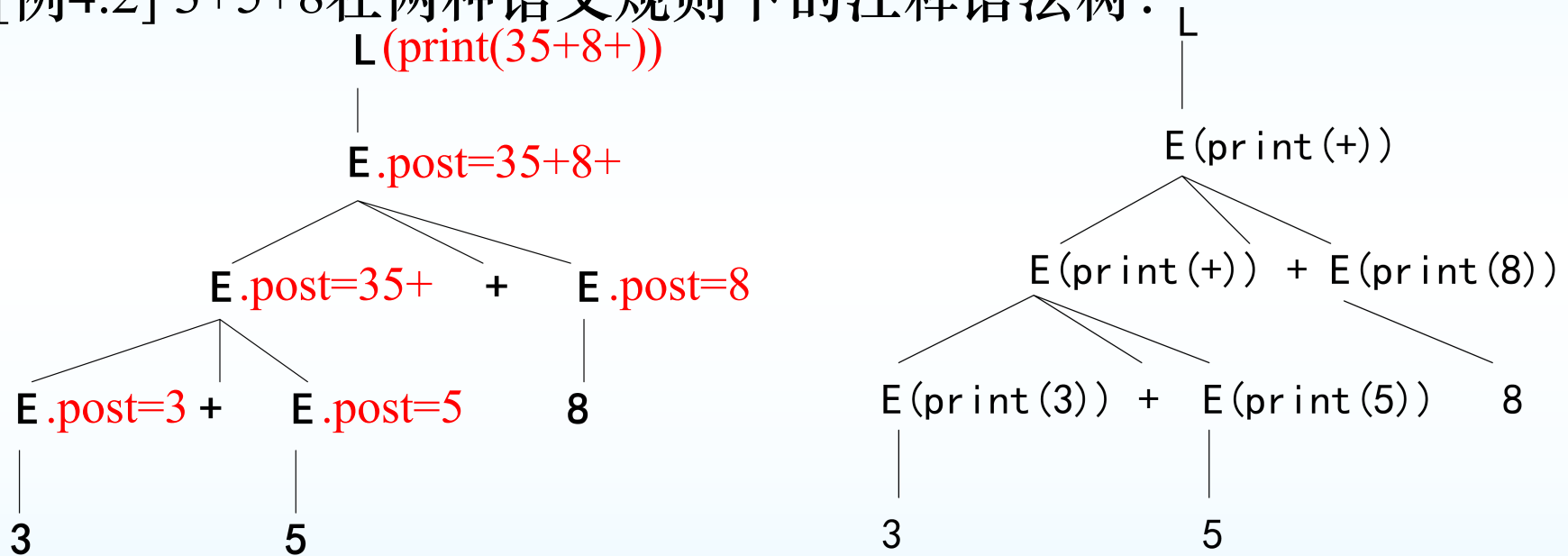


4.1 语法制导翻译简介

3. 属性作为语法树的注释

将属性附着在语法树对应文法符号上，形成**注释语法树**。

[例4.2] $3+5+8$ 在两种语义规则下的注释语法树：



产生式	语法制导定义	翻译方案2
$L \rightarrow E$	$\text{print}(E.\text{post})$	
$E \rightarrow E1 + E2$	$E.\text{post} := E1.\text{post} \parallel E2.\text{post} \parallel '+'$	$\text{print}(+)$
$E \rightarrow \text{num}$	$E.\text{post} := \text{num.lexval}$	$\text{print}(\text{lexval})$



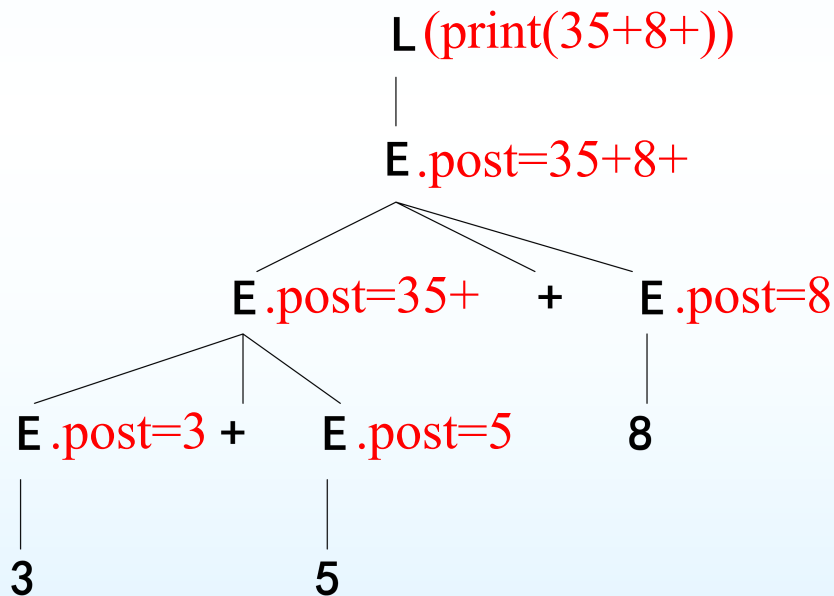
4.1 语法制导翻译简介

4. 注释语法树上看继承属性与综合属性

✧ 继承属性是自上而下计算的

✧ 综合属性是自下而上计算的

提醒：除非特别提醒，本章讨论的语法制导翻译是**综合属性**。





4.1 语法制导翻译简介

4.1.4 LR分析翻译方案的设计

LR分析中的语法制导翻译实质上是对LR语法分析的扩充：

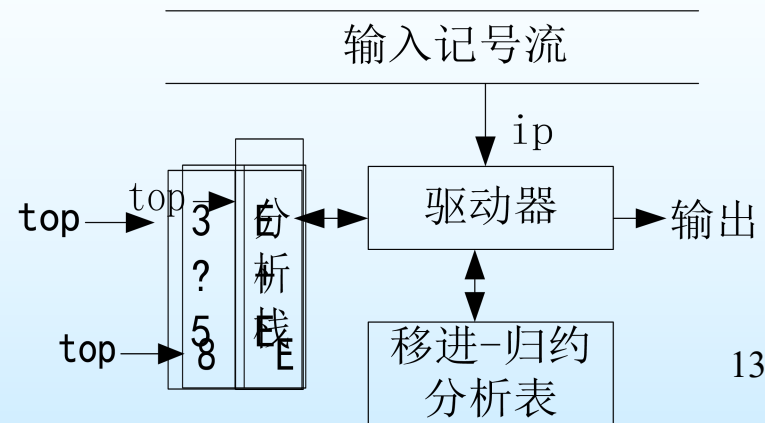
- ① **扩充LR分析器的功能**：当执行归约产生式的动作时，也执行产生式对应的语义动作。由于是**归约时执行语义动作**，限制语义动作仅能放在产生式右部的最右边；
- ② **扩充分析栈**：增加一个与分析栈并列的**语义栈**，用于存放分析栈中文法符号所对应的属性值。

例如： $E \rightarrow E1 + E2$ $val[top-2] := val[top] + val[top-2]; top := top-2;$

对于表达式 $5+3$

当归约为左部 E 时

同时也得到了值 8





[例4.3] 计算表达式值的语法制导翻译。以 $3+5*8$ 为例。

产生式

语法制导定义

翻译方案(用数组实现语义栈)

$L \rightarrow E$	$\text{print}(E.\text{val})$	$\text{print}(\text{val}[\text{top}]);$
$E \rightarrow E1 + E2$	$E.\text{val} := E1.\text{val} + E2.\text{val};$	$\text{val}[\text{top}-2] := \text{val}[\text{top}] + \text{val}[\text{top}-2]; \text{top} := \text{top}-2;$
$E \rightarrow E1 * E2$	$E.\text{val} := E1.\text{val} * E2.\text{val};$	$\text{val}[\text{top}-2] := \text{val}[\text{top}] * \text{val}[\text{top}-2]; \text{top} := \text{top}-2;$
$E \rightarrow n$	$E.\text{val} := n.\text{lexval};$	$\text{val}[\text{top}] := \text{lexval};$

分析栈 语义栈 输入 语法分析动作 语义动作

#	#	3+5*8#	shift	
#n	#3	+5*8#	$E \rightarrow n,$	$\text{val}[\text{top}] := \text{lexval};$
#E	#3	+5*8#	shift	
#E+	#3?	5*8#	shift	
#E+n	#3?5	*8#	$E \rightarrow n,$	$\text{val}[\text{top}] := \text{lexval};$
#E+E	#3?5	*8#	shift	
#E+E*	#3?5?	8#	shift	
#E+E*n	#3?5?8	#	$E \rightarrow n,$	$\text{val}[\text{top}] := \text{lexval};$
#E+E*E	#3?5?8	#	$E \rightarrow E1 * E2,$	$\text{val}[\text{top}-2] := \text{val}[\text{top}] * \text{val}[\text{top}-2]; \text{top} := \text{top}-2$
#E+E	#3?40	#	$E \rightarrow E1 + E2,$	$\text{val}[\text{top}-2] := \text{val}[\text{top}] + \text{val}[\text{top}-2]; \text{top} := \text{top}-2$
#E	#43	#	acc	$\text{print}(\text{val}[\text{top}]);$



4.2 中间代码简介

1. 编译器各阶段的完整输出，均可以被认为是源程序的某种**中间表示**。
2. 本节讨论的是**中间代码生成器**输出的中间表示，称之为**中间代码**。
3. 中间代码实际上应起一个编译器前端与后端分水岭的作用。
4. 要求中间代码具有如下特性，以**便于编译器的开发、移植和代码优化**：
 - ① 便于语法制导翻译；
 - ② 既与机器指令的结构相近，又与具体机器无关。
5. 中间代码的主要形式：后缀式、三地址码、树等。



4.2 中间代码简介

4.2.1 后缀式（逆波兰式）

特点：操作数在前，操作符紧随其后；

无需用括号限制运算的优先级和结合性。

算法4.1 后缀式计算

给定某个后缀式，采用下述过程进行计算，最终结果留在栈中。

```
x := first_token;
while not end_of_exp
loop      if x in operands
          then push x;           -- 操作数进栈
          else pop(operands);    -- 算符，弹出操作数
              push(evaluate);    -- 计算，将结果进栈
          end if;
      next(x);
end loop;
```

后缀式不需要括号；
事先知道运算符的元数



4.2 中间代码简介

```
loop   if  x in operands
      then  push x;           -- 操作数进栈
      else  pop(operands); -- 算符，弹出操作数
           push(evaluate); -- 计算，将结果进栈
      end if;
      next(x);
end loop;
```

算术表达式 $3+5+8$ 的后缀式为 $35+8+$ 。

#	35+8+#	push(3)
#3	5+8+#	push(3)
#35	+8+#	pop(3和5), push(3+5)
#8	8+#	push(8)
#88	+8	pop(8和8), push(8+8)
#16	#	



4.2 中间代码简介

后缀式并不局限于二元运算的表达式，可以推广到任何语句，遵守操作数在前，操作符紧跟其后的原则即可。

对语句：**if e then x else y**

后缀式可以写为：**e x y if-then-else** (1)

此时e、x和y均需计算。

实际上，根据条件e的取值，x和y不用都计算：

e p1 jez x p2 jump p1: y p2: (2)

其中：p1和p2分别是标号；

p1 jez表示e的结果为0（假）则转向p1；

p2 jump表示无条件转向p2。

与(1)比较，(2)中的if-then-else被分解，首先计算e，根据e的结果是否为真，决定计算x还是计算y。



4.2 中间代码简介

特点：与汇编指令相近，但与机器无关，便于进行与机器无关的代码优化，是最终生成目标代码的理想中间代码形式。

4.2.2 三地址码

1. 三地址码的直观表示

形式(部分): $\text{result} := \text{arg1 op arg2}$ 或
 $\text{result} := \text{op arg1}$ 或
 op arg1

含义：结果存放在result中的二元运算 arg1 op arg2
结果存放在result中一元运算 op arg1
一元运算 op arg1

赋值句 $x := a + b * c$ 的三地址码序列 (T1, T2 是临时变量)

$T1 := b * c$

$T2 := a + T1$

$x := T2$

注：每个三地址码至多进行一个二元运算



4.2 中间代码简介

2. 三地址码的种类

序号	三地址码	四元式
(1)	$x := y \text{ op } z$	(op, y, z, x)
(2)	$x := \text{op } y$	$(\text{op}, y, , x)$
(3)	$x := y$	$(:=, y, , x)$
(4)	goto L	$(j, , , L)$
(5)	if x goto L	$(\text{jnz}, x, , L)$
(6)	if $x \text{ relop } y$ goto L	(jrelop, x, y, L)
(7)	param x	$(\text{param}, , , x)$
(8)	call n, P	$(\text{call}, n, , P)$
(9)	return y	$(\text{return}, , , y)$
(10)	$x := y[i]$	$(=[], y[i], , x)$
(11)	$x[i] := y$	$([]=, y, , x[i])$
(12)	$x := \&y$	$(=\&, y, , x)$
(13)	$x := *y$	$(=* , y, , x)$
(14)	$*x := y$	$(*=, y, , x)$



4.2 中间代码简介

3. 三地址码的实现：三元式与四元式

① 三元式

三元式 (i) (op, arg1, arg2) 三地址码 (i) := arg1 op arg2

[例4.5] 表达式 $x := a + b * c$ 的三元式序列：

(1) (*, b, c)

(2) (+, a, (1))

(3) (:=, x, (2))

序号是三元式中的一部分

序号的双重含义：既代表此三元式，又代表三元式存放的结果

存放方式：数组，序号由三元式在数组中的位置(下标)决定。

弱点：给代码的优化带来困难。因为代码优化常使用的方法是删除或移动某些代码，而一旦进行了代码的删除或移动，则表示某些三元式的序号会发生变化，从而使得其他三元式中对原序号的引用失效。



4.2 中间代码简介

语法制导翻译设计的基本步骤

1. 文法符号属性的设计
2. 必要的基本操作（函数等）的设计
3. 语义规则的设计

② 三元式的语法制导翻译

1. **属性** .code: 表示三元式代码，值可以是标识符的存储位置或三元式表中的位置（序号）；
2. **属性** .name: 给出标识符的名字；
3. **函数** trip(op, arg1, arg2): 生成一个三元式，返回三元式的序号；
4. **函数** entry(id.name): 返回标识符在符号表中的位置或存储位置。为了直观，在三元式中仍用标识符名字表示。



4.2 中间代码简介

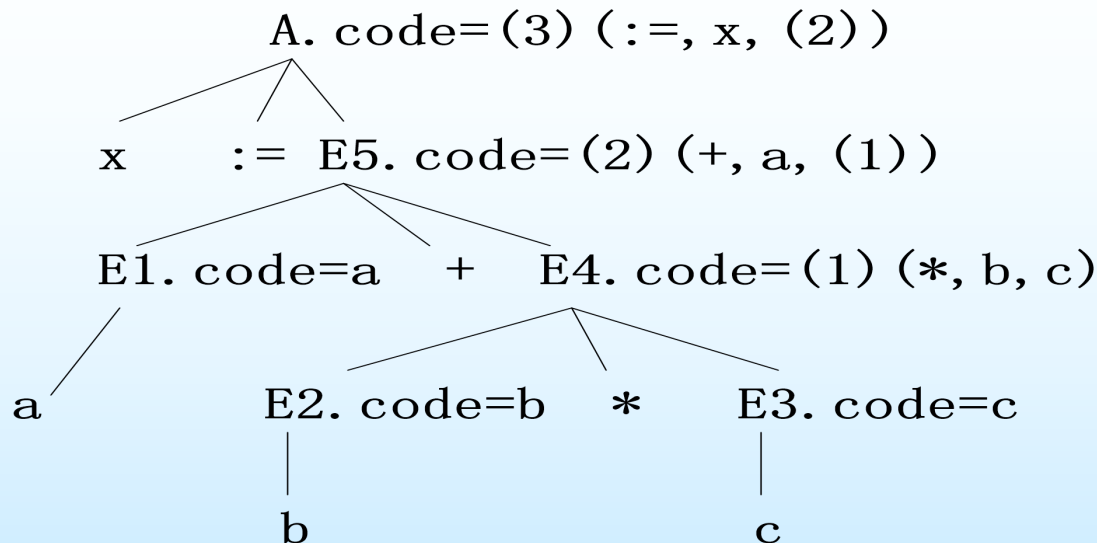
产生式

- (1) $A \rightarrow id := E$
- (2) $E \rightarrow E1 + E2$
- (3) $E \rightarrow E1 * E2$
- (4) $E \rightarrow (E1)$
- (5) $E \rightarrow -E1$
- (6) $E \rightarrow id$

语义规则

- $\{A.code := trip(:=, entry(id.name), E.code)\}$
- $\{E.code := trip(+, E1.code, E2.code)\}$
- $\{E.code := trip(*, E1.code, E2.code)\}$
- $\{E.code := E1.code\}$
- $\{E.code := trip(@, E1.code,)\}$
- $\{E.code := entry(id.name)\}$

[例4.6] 生成 $x := a + b * c$ 的三元式序列(LR分析)



- (1) $(*, b, c)$
- (2) $(+, a, (1))$
- (3) $(:=, x, (2))$



4.2 中间代码简介

③ 四元式

四元式是对三元式的改进，将表示计算结果的三元式序号用一个**显式的变量**表示，从而避免了**三元式的值与三元式在三元组中的位置**相关的弱点。

四元式的语法：(op, arg1, arg2, result)

所表示的计算：result := arg1 op arg2

四元式的特点：

1. 四元式与三元式的唯一区别：将由**序号**所表示的运算结果改为由**临时变量**来表示。
2. 此改进使得四元式的运算结果与其位置无关，为代码优化提供了极大方便：可以移动四元式而不影响运算结果。
3. 三地址码与四元式形式的保持一致。**四元式的种类**

三元式

(i) (op, arg1, arg2)

(i):= arg1 op arg2



4.2 中间代码简介

四元式的种类

序号	三地址码	四元式
(1)	$x := y \text{ op } z$	(op, y, z, x)
(2)	$x := \text{op } y$	$(\text{op}, y, , x)$
(3)	$x := y$	$(:=, y, , x)$
(4)	goto L	$(j, , , L)$
(5)	if x goto L	$(\text{jnz}, x, , L)$
(6)	if $x \text{ relop } y$ goto L	(jrelop, x, y, L)
(7)	param x	$(\text{param}, , , x)$
(8)	call n, P	$(\text{call}, n, , P)$
(9)	return y	$(\text{return}, , , y)$
(10)	$x := y[i]$	$(=[], y[i], , x)$
(11)	$x[i] := y$	$([]=, y, , x[i])$
(12)	$x := \&y$	$(=\&, y, , x)$
(13)	$x := *y$	$(=* , y, , x)$
(14)	$*x := y$	$(*=, y, , x)$



4.2 中间代码简介

④ 四元式的语法制导翻译

1. **属性**.code: 表示存放运算结果的变量, 或标识符的存储位置;
2. **属性**.name: 给出标识符的名字;
3. **函数**newtemp: 返回一个新的临时变量, 如T1, ...等;
4. **过程**emit(op, arg1, arg2, result): 生成一个四元式, 若为一元运算, 则arg2可空;
5. **函数**entry(id.name): 返回标识符在符号表中的位置或存储位置。

产生式与语义规则:

- (1) $A \rightarrow id := E$ $\{emit(:=, E.code, , entry(id.name))\}$
- (2) $E \rightarrow E1 + E2$ $\{E.code := newtemp; emit(+, E1.code, E2.code, E.code)\}$
- (3) $E \rightarrow E1 * E2$ $\{E.code := newtemp; emit(*, E1.code, E2.code, E.code)\}$
- (4) $E \rightarrow (E1)$ $\{E.code := E1.code\}$
- (5) $E \rightarrow -E1$ $\{E.code := newtemp; emit(@, E1.code, , E.code)\}$
- (6) $E \rightarrow id$ $\{E.code := entry(id.name)\}$



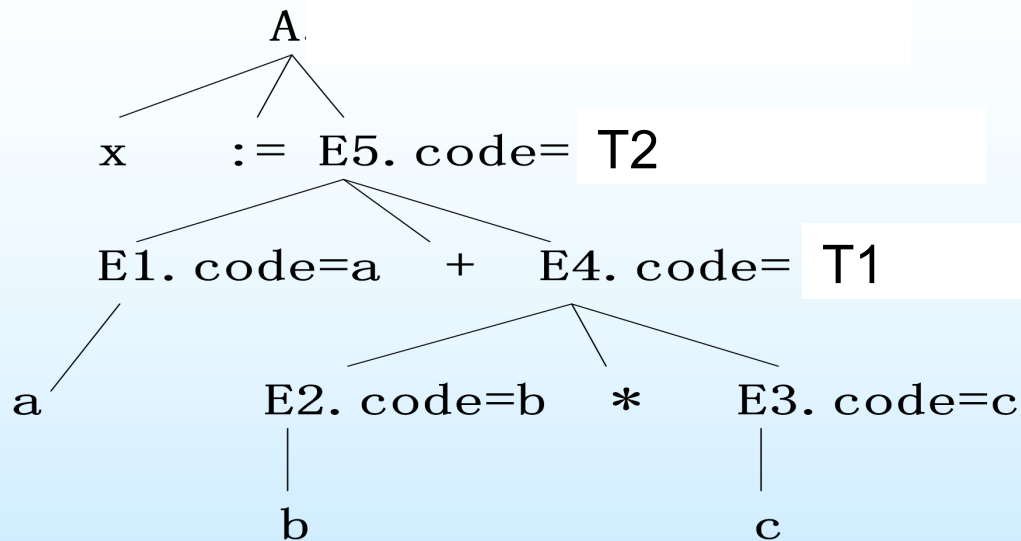
4.2 中间代码简介

产生式

语义规则

- (1) $A \rightarrow id := E$ $\{emit(:=, E.code, , entry(id.name))\}$
- (2) $E \rightarrow E1 + E2$ $\{E.code := newtemp; emit(+, E1.code, E2.code, E.code)\}$
- (3) $E \rightarrow E1 * E2$ $\{E.code := newtemp; emit(*, E1.code, E2.code, E.code)\}$
- (4) $E \rightarrow (E1)$ $\{E.code := E1.code\}$
- (5) $E \rightarrow -E1$ $\{E.code := newtemp; emit(@, E1.code, , E.code)\}$
- (6) $E \rightarrow id$ $\{E.code := entry(id.name)\}$

[例4.6] 生成 $x := a + b * c$ 的四元式序列(LR分析)



$(*, b, c, T1)$
 $(+, a, T1, T2)$
 $(:=, T2, , x)$



4.2 中间代码简介

4.2.3 图形表示

1. 树作为中间代码

语法树真实反映句子结构，对语法树稍加修改，即可以作为中间代码的一种形式(抽象语法树)。(抽象语法树不含非终结符)

[例4.8] 赋值句 $x := (a+b) * (a+b)$ 的树的中间代码表示：

三元式：

(1)(+, a, b)

(2)(+, a, b)

(3)(*, (1), (2))

(4)(:=, x, (3))

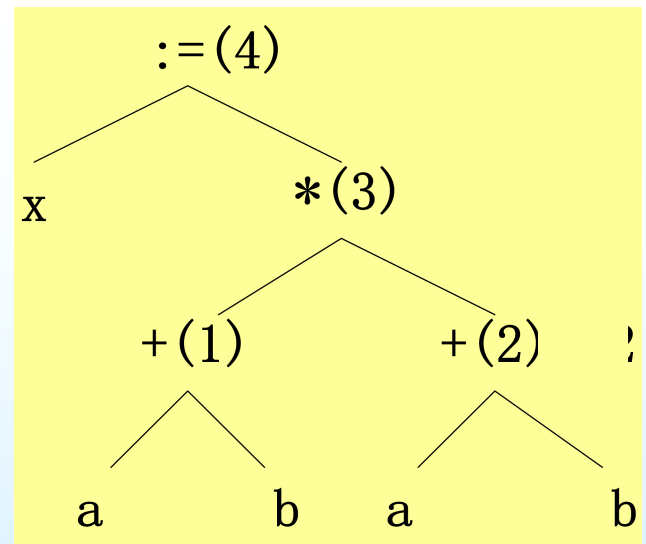
四元式：

(+, a, b, T1)

(+, a, b, T2)

(*, T1, T2, T3)

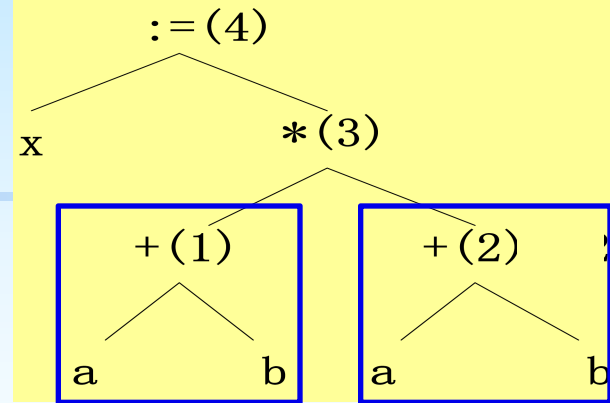
(:=, T3, , x)



抽象语法树：(1)根和内部结点表示一个运算；

(2)叶子表示操作数；(3)运算顺序由附加在根和内部结点上的序号表示。

4.2 中间代码简介



2. 树的语法制导翻译

- ① **属性**`nptr`: 指针, 用来指向树节点;
- ② **函数**`mknode(op, nptr1, nptr2)`: 生成一个**根或内部节点**, 节点数据是`op`, 左右孩子分别是`nptr1`和`nptr2`指向的子树。若仅有一个孩子, 则`nptr2`为空;
- ③ **函数**`mkleaf(node)`: 生成一个**叶子节点**。

(1) $A \rightarrow \text{id} := E$	$\{A.\text{nptr} :=$ $\text{mknode}(:=, \text{mkleaf}(\text{entry}(\text{id.name})), E.\text{nptr})\}$
(2) $E \rightarrow E1 + E2$	$\{E.\text{nptr} := \text{mknode}(+, E1.\text{nptr}, E2.\text{nptr})\}$
(3) $E \rightarrow E1 * E2$	$\{E.\text{nptr} := \text{mknode}(*, E1.\text{nptr}, E2.\text{nptr})\}$
(4) $E \rightarrow (E1)$	$\{E.\text{nptr} := E1.\text{nptr}\}$
(5) $E \rightarrow - E1$	$\{E.\text{nptr} := \text{mknode}(@, E1.\text{nptr},)\}$
(6) $E \rightarrow \text{id}$	$\{E.\text{nptr} := \text{mkleaf}(\text{entry}(\text{id.name}))\}$

三元式、四元式与树的语义规则设计是相似的。

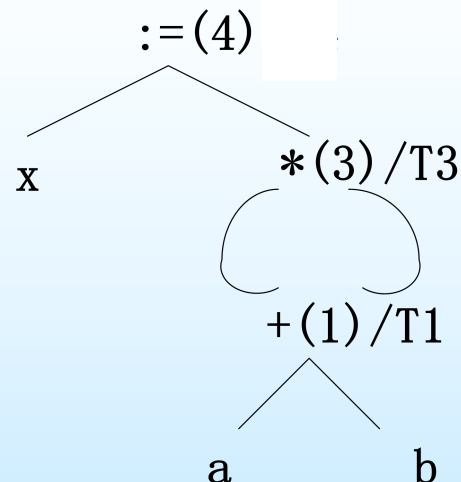
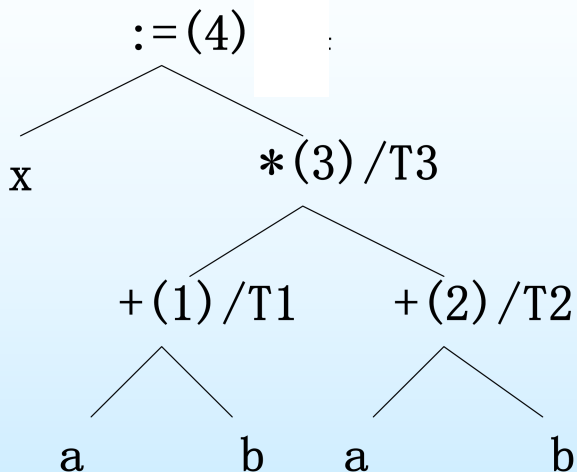


4.2 中间代码简介

3. 树的优化表示—**DAG (Directed Acyclic Graph, 有向无环图)**

如果树上若干个节点有完全相同的孩子，则这些节点可以指向同一个孩子，形成一个有向无环图(Directed Acyclic Graph, DAG)，与树的唯一区别是多个父亲可以共享同一个孩子，从而达到资源(运算、代码等)共享的目的。

DAG的语法制导翻译与树的语法制导翻译相似，仅需要在mknode和mkleaf中增加相应的**查询功能**：先查看所要构造的节点是否已经存在，若存在则无需构造新的节点，直接返回指向已存在节点的指针即可。





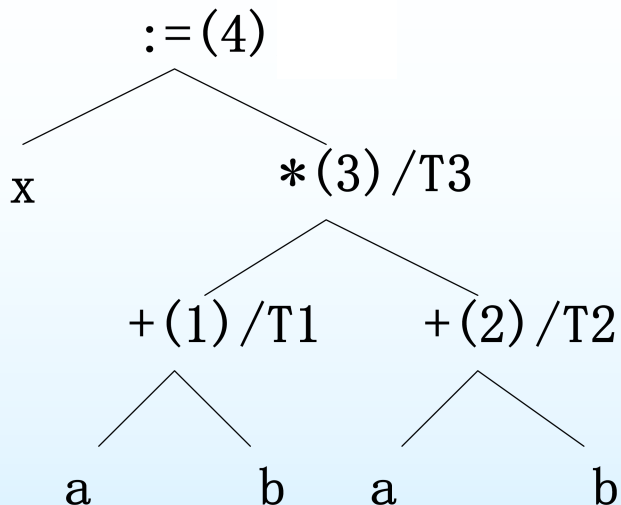
4.2 中间代码简介

4. 树与其他中间代码的关系

树表示的中间代码与后缀式和三地址码之间有内在联系：

- ※ 对树进行深度优先后序遍历，得到的线性序列就是后缀式，或者说后缀式是树的一个线性化序列；
- ※ 树的“每个内部节点和它的孩子”对应一个三元式或四元式。

[例4.9] 赋值句 $x := (a+b) * (a+b)$ 的抽象语法树：



后缀式： $xab+ab+*:=$

三元式：

(1)(+, a, b)
(2)(+, a, b)
(3)(*, (1), (2))
(4)(:=, x, (3))

四元式：

(+, a, b, T1)
(+, a, b, T2)
(*, T1, T2, T3)
(:=, T3, , x)

现代的编译器基础架构均用抽象语法树作为中间表示。



4.3 符号表简介

符号表的作用：

连接声明与引用的桥梁。一个名字在声明时，相关信息被填写进符号表；而在引用时，根据符号表中的信息生成相应的中间代码序列。

符号表设计的基本要求：合理存放信息、快速准确查找

1. 有效记录各类符号信息，便于查找、插入、删除和修改；
2. 符号表的信息组织与符号表的数据结构的安排对于编译的效率有重大影响。合理组织符号表的内容，以适应编译不同阶段的需求；
3. 符号表的存储空间无论多大，都会有溢出的可能，因此**符号表的空间存储应该是可以动态扩充的。**



4.3 符号表简介

4.3.1 符号表条目

- ※ 逻辑上讲：每个声明的名字在符号表中占据一栏，称为一个**条目**，用于存放名字的相关信息。
- ※ 符号表中的内容：保留字、**标识符**、特殊符号(包括算符、分隔符等)等。
- ※ 多个子表：不同类别的符号可以存放在不同的子表中，如变量名表、过程名表、保留字表等。
- ※ 存放方式：**名字+属性**。

组合关键字：`int x;`
`{int x;`
`struct x { float y, z; };`
`}`

一个名字 x 在同一作用域中允许有多个的声明，则引用时需要根据上下文确定 x 属于哪个对象。为简化编译，大多数语言在语法上不允许这样的声明

C符号表的组合关键字至少包括：**名字+作用域+类型**



4.3 符号表简介

4.3.2 构成名字的字符串的存储

定长数据采用直接存放，变长数据采用间接存放。

名字（直接存储）

sort

a

readarray

draw_a_red_line_for_object_a

名字（间接存储）

101 (或101/4)

106 (或105/1)

108 (或106/9)

118 (或115/28)

sorta readarray draw_a_red_line_for_object_a#

↑101

属性

proc, ...

int, ...

proc, ...

boolean, ...

属性

proc, ...

int, ...

proc, ...

boolean, ...

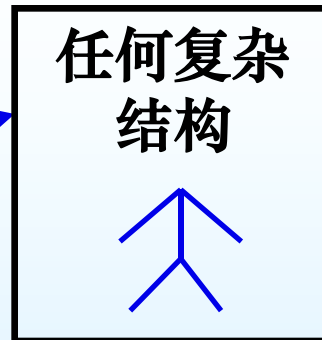


4.3 符号表简介

间接存储的特点：

1. 间接存储的方法实际上解决了复杂信息的存储问题；
2. 将其推广到属性，则任何一个复杂的属性，均可以为其另辟空间；
3. 空间本身可以是任何复杂结构；
4. 指向空间的指针存放于此属性在符号表中的对应位置。

关键字	属性
x	
...	



```
proc sort;  
  a : array[10] of int;  
  x : int;  
  proc readarray;  
    i : int;  
    read(a);  
  readarray
```



4.3 符号表简介

4.3.3 名字的作用域

程序设计语言中一个名字可以出现在不同的范围内，并且可以具有不同的意义。

1. **范围的两种划分方式**：并列的和嵌套的。
2. **不同的语言采用不同的方式**：如Pascal的**过程定义**可以是嵌套的，而C/C++的**过程定义**是并列的，但是C/C++允许**程序块(block)**是嵌套的，每个程序块的范围用{}界定。
3. **名字的作用域**：名字在哪个范围内起作用。并列的两个范围内的名字作用域互不相干，但是在嵌套的两个范围内的名字，其作用域的问题就**需要制定规则**来限定，以使得**任何一个名字在任何范围内涵义都是无二义的**。
4. **名字的作用域规则**：规定一个名字在什么样的范围内应该表示什么意义的原则。



4.3 符号表简介

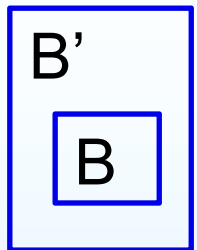
1. 静态作用域规则(static-scope rule)——总方针

编译时就可以确定名字的作用域，即仅从静态读程序就可确定名字的作用域。

2. 最近嵌套规则(most closely nested)——总方针下的具体规则

以程序块为例，也适用于过程。

- ① 程序块B中声明的作用域包括B；
- ② 如果名字x不在B中声明，那么B中x的出现是在外围程序块B'的x声明的作用域中，其中B'满足以下条件：
 - a) B'有x的声明，并且
 - b) B'比其它任何含x声明的程序块更接近被嵌套的B



通俗地讲，在名字引用处**从内向外**看，它处在所遇到的**第一个该名字声明的作用域**。根据名字的引用位置和名字作用域规则，判断名字的声明位置。



[例4.10] 符合作用域规则的C++程序。

void main()

```
{  int a=0, b=0;
    {  int b=1;
        {  int a=2, c=4, d=5;
            printf("%d %d\n", a, b);
        }
        {  int b=3;
            printf("%d %d\n", a, b);
        }
        printf("%d %d\n", a, b);
    }
    printf("%d %d\n", a, b);
}
```

/* B0层 */

/* B1层，被B0嵌套 */

/* B2层，被B1嵌套 */

/* 结果为： 2， 1 */

/* B3层，与B2并列 */

/* 结果为： 0， 3 */

/* 结果为： 0， 1 */

/* 结果为： 0， 0 */

声 明

作用域

int a=0

B0-B2

int b=0

B0-B1

int b=1

B1-B3

int a=2

B2

int b=3

B3



4.4 声明语句的翻译

声明语句的作用是为可执行语句提供信息，以便于其执行。对声明语句的处理，主要是将所需要的信息正确地填写进合理组织的符号表中。

4.4.1 变量的声明

1. 变量的类型定义与声明

类型定义为编译器提供存储空间大小的信息，**变量声明**为变量分配存储空间。

- ※ **简单数据类型**的存储空间是已经确定的，如integer可以占4个字节，real可以占8个字节，char可以占1个字节等
- ※ **组合数据类型**变量的存储空间，需要编译器根据程序员提供的信息计算而定，例如数组，结构，记录等

组合数据的类型定义和变量声明有两种情况：定义与声明在一起，定义与声明分离。



4.4 声明语句的翻译

[例] 在Pascal程序中的类型定义与变量声明:

先类型定义后变量声明

```
type    player = array[1..2] of integer;  
        matrix = array[1..24] of char;  
  
var      c, p   : player;  
        winner : boolean;  
        display : matrix;  
        movect  : integer;
```

类型定义与变量声明同时

```
var      c, p   : array[1..2] of integer;  
        display : array[1..24] of char;
```

强调:

1. 简单变量声明中类型是预定义的;
2. 组合变量声明中类型需自己定义。(定义的两种形式)



4.4 声明语句的翻译

2. 变量声明的语法制导翻译

① 变量声明的文法:

$D \rightarrow D ; D$ (1)

$\mid id : T$ (2)

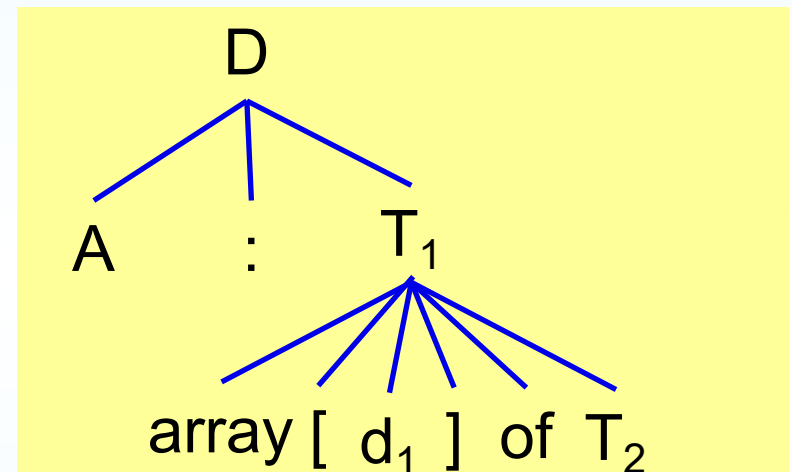
$T \rightarrow int$ (3)

$\mid real$ (4)

$\mid array [num] of T$ (5)

$\mid ^T$ (6)

此多维数组以行为主存储，可看作有 d_1 个元素的一维数组，每个元素又是一个 $n-1$ 维的数组；依此类推。



(5)是**数组类型的声明**，其中的数组元素个数由num表示，如num可以是5或10等，它等价于1..5或1..10。

(6)是**指针类型的声明**，其**宽度(大小)**是一个常量。数组元素的类型和指针所指对象的类型可以是任意合法类型。

声明多维数组: $A : array [d1] of array [d2] of \dots array [dn] of int_{41}$



4.4 声明语句的翻译

② 填写符号表信息的语法制导定义

1. **全程量**offset: 记录当前符号存储的偏移量, 初值设为0
2. **属性**.type和.width: 变量的类型和所需占据的存储空间
3. **过程**enter(name, type, offset): 为type类型的变量name建立符号表条目, 并为其分配存储空间(位置)offset

(1) $D \rightarrow D; D$

(2) $D \rightarrow id:T \quad \{ \text{enter}(id.name, T.type, offset);$
 $\quad \quad \quad offset := offset + T.width; \}$

(3) $T \rightarrow int \quad \{ T.type := integer; T.width := 4; \}$

(4) $T \rightarrow real \quad \{ T.type := real; T.width := 8; \}$

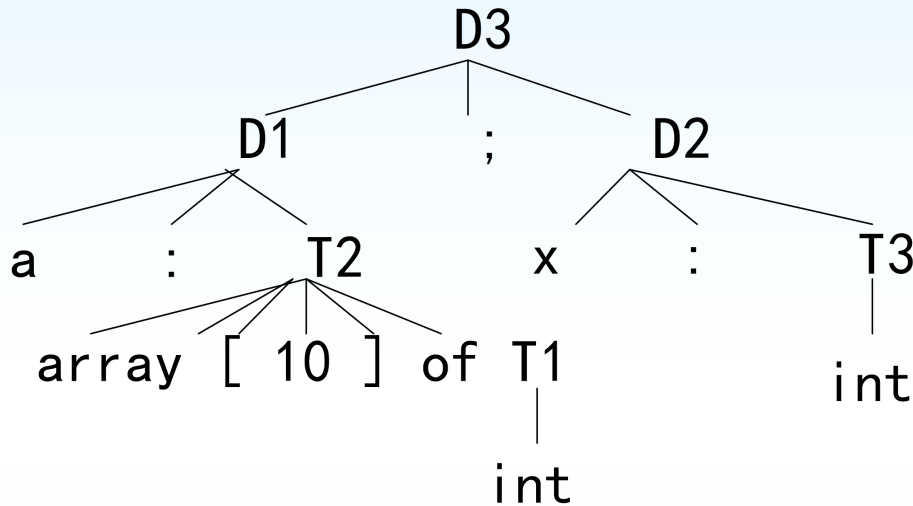
(5) $T \rightarrow array [num] \text{ of } T1 \quad \{ T.type := array(num.val, T1.type);$
 $\quad \quad \quad T.width := num.val * T1.width; \}$

(6) $T \rightarrow ^T1 \quad \{ T.type := pointer(T1.type); T.width := 4; \}$



4.4 声明语句的翻译

[例] 声明的语法制导翻译: `a : array [10] of int ; x : int`



生成的符号表:

在存储空间
中的偏移量

a	array (10, integer)	0
x	integer	40
		

序号 归约使用的产生式

(1) $T1 \rightarrow \text{int}$

(2) $T2 \rightarrow \text{array}[\text{num}] \text{ of } T1$

(3) $D1 \rightarrow \text{id} : T2$

(4) $T3 \rightarrow \text{int}$

(5) $D2 \rightarrow \text{id} : T3$

语义处理结果

$T1.\text{type} = \text{integer}$ $T1.\text{width} = 4$

$T2.\text{type} = \text{array}(10, \text{integer})$

$T2.\text{width} = 10 * 4 = 40$

$\text{enter}(a, \text{array}(10, \text{integer}), 0)$ $\text{offset} = 40$

$T3.\text{type} = \text{integer}$ $T3.\text{width} = 4$

$\text{enter}(x, \text{integer}, 40)$ $\text{offset} = 44$



4.4 声明语句的翻译

4.4.2 过程的定义与声明

- ✧ 过程（procedure）：过程头+过程体；
- ✧ 过程的三种形式：过程定义、过程声明和过程调用

Ada过程定义：

```
procedure swap(x,y:in out integer) is
```

```
    temp : integer;
```

```
begin temp := x; x := y; y := temp; end swap;
```

-- 规格说明(过程头)

-- 体中的声明

-- 可执行语句序列

声明与引用：

```
procedure swap(x, y: in out integer);
```

```
swap(a, b);
```

-- 过程声明

-- 过程调用

- ✧ 先声明后引用原则

过程定义可以写在对它的引用之后或引用时看不到的地方。在这样的情况下，引用前必须先声明。而若引用前已定义，则声明可省略，因为**定义已包括了声明**。



4.4 声明语句的翻译

1. 左值与右值

- ① 形式上，出现在赋值号左边和右边的对象称为左值和右值
- ② 实质上，左值必须具有存储空间，右值可以仅是一个值，而没有存储空间。
- ③ 形象地讲，左值是容器，右值是内容。

```
(1) const two = 2;           -- 声明一个值为2的常量two
```

(2) `x : integer;` **-- 声明一个类型为整型的变量x**

```
(3) function max(a, b : integer) return integer;  
    -- 声明一个返回值类型是整型数的函数max
```

(4) `x := two;` **-- x当前值为2** 左值: 变量

(5) `x := two + x;` **-- x当前值变为4** 右值：字面量、常量、

(6) `x := max(two,x)+x;` -- **x当前值变为8** 函数返回值、
(7) `4 := x;` -- **字面量不能作为左值** 表达式、变量的值

(7) `4 := x;` **-- 字面量不能作为左值**

```
(8) two := x;           -- 常量不能作为左值
```

(9) `max(two,x) := two; -- 函数返回值不能作为左值`

(10) `x+two := x+two;` **-- 表达式的值不能作为左值**



4.4 声明语句的翻译

2. 参数传递

* 形参与实参

- ① 定义/声明时的参数称为形参(parameter或formal parameter)
- ② 调用时的参数称为实参(argument或actual parameter)

* 常见的参数传递形式：（不同的语言提供不同的形式）

- ① 值调用 (call-by-value)
- ② 引用调用 (call-by-reference)
- ③ 复写—恢复 (copy-in/copy-out)
- ④ 换名调用 (call-by-name)

* 参数传递方法的实质：

实参是代表左值、右值、还是实参本身的正文。



4.4 声明语句的翻译

① 值调用

值调用的特点：调用时首先计算实参，把它的**右值**传递给被调用过程。过程内部对参数的修改，不影响实参对象。

实参的特点：**任何可以作为右值的对象**均可作为实参，所以值调用的实参不一定有存储空间。

C语言采用的参数传递方式是值调用。

值调用的参数传递和过程内对参数的使用原则：

1. 过程定义时，形参被当作局部名看待，并在过程内部为形参分配存储单元；
2. 调用过程前，首先计算实参并将值（实参的右值）放入形参的存储单元；
3. 过程内部对形参单元中的数据**直接**访问。



4.4 声明语句的翻译

值调用举例：Pascal程序

```
program reference( input, output);  
  var a, b : integer;  
  procedure swap(x, y : integer);  
    var temp : integer;  
    begin temp:=x; x:=y; y:=temp end;  
begin  
  a:=1;  b:=2;  
  swap(a, b);  
  writeln('a=', a); writeln('b=', b)  
end.
```

运行结果： a=1 b=2

// 等价的C程序

```
void swap(int x, int y)  
{  int temp;  
   temp=x; x=y; y=temp;  
}  
  
void main ()  
{  int a=1, b=2;  
   swap(a, b);  
   printf("a=%d,b=%d\n",a,b);  
}
```



4.4 声明语句的翻译

② 引用调用

引用调用的特点：调用时首先计算实参的地址，将此地址传递给被调用过程。过程内部对形参的修改，等价于直接对实参的修改。

实参的特点：必须是左值。

C语言不提供引用调用方式。

引用调用的参数传递和过程内对参数的使用原则：

1. 过程定义时，形参被当作局部名看待，并在过程内部为形参分配存储单元；
2. 调用过程前，将作为实参的变量的地址放进形参的存储单元；
3. 过程内部把形参单元中的数据当作地址，进行间接访问



4.4 声明语句的翻译

引用调用举例：Pascal程序

```
program reference( input, output);  
  var a, b : integer;  
  procedure swap( var x, y : integer);  
    var temp : integer;  
    begin temp:=x; x:=y; y:=temp end;  
begin  
  a:=1;  b:=2;  
  swap(a, b);  
  writeln('a=', a); writeln('b=', b)  
end.
```

运行结果： a=2 b=1

```
// 等价的C++程序(引用调用)  
#include <iostream.h>  
void swap(int &x, int &y)  
{  int temp;  
   temp=x; x=y; y=temp;  
}  
void main ()  
{  int a(1), b(2);  
   swap(a, b);  
   cout<<"a="<<a<<"b="<<b;  
}
```



4.4 声明语句的翻译

引用调用举例：Pascal程序

```
program reference( input, output);  
  var a, b : integer;  
  procedure swap( var x, y : integer);  
    var temp : integer;  
    begin temp:=x; x:=y; y:=temp end;  
begin  
  a:=1; b:=2;  
  swap(a, b);  
  writeln('a=', a); writeln('b=', b)  
end.
```

运行结果： a=2 b=1

// 等价的C程序(值调用模拟引用调用)

```
void swap(int *x, int *y)  
{  
  int temp;  
  temp=*x; *x=*y;  
  *y=temp;  
}  
  
void main ()  
{  
  int a=1, b=2;  
  swap(&a, &b);  
  printf("a=%d,b=%d\n",a,b);  
}
```



4.4 声明语句的翻译

③ 复写—恢复

引用调用的副作用

```
int a=2;
```

```
void add_one(int &x){ a=x+1; x=x+1; }
```

```
void main ()
```

```
{ cout<<"before: a="<<a<<endl;
```

```
  add_one(a);
```

```
  cout<<"after: a="<<a<<endl;
```

```
}
```

本意： a=3 **结果：** a=4

原因： 实参与非本地量共用一个存储空间，使得在过程内改变参数值的同时，也改变了非本地量的值。



4.4 声明语句的翻译

复写-恢复的特点：(值调用和引用调用的结合)

1. 过程内部对参数的修改不直接影响实参，避免了副作用；
2. 返回时将形参内容恢复给实参，实现了参数的返回。

实参的特点：必须是左值。

参数传递和过程内对参数的使用原则：

1. 过程定义时形参被当作局部名，在过程内部为形参分配单元(**复写**)；
2. 调用过程前，计算实参并将右值放入形参的存储单元；
3. 过程内部对形参单元中的数据**直接**访问；
4. **过程返回前，将形参右值放回实参的存储单元(恢复)。**

注：虽然调用时传递的是右值，但是返回时需要实参有对应的存储空间，因此要求实参应该是变量而不是表达式或常量。



4.4 声明语句的翻译

Ada语言支持复写-恢复的参数传递方式

复写-恢复举例：

```
procedure test is                                     -- Ada源程序
    a : integer;
    procedure add_one(x:in out integer);             -- 复写-恢复调用
    begin a:=x+1; x:=x+1; end add_one;
begin a:=2; add_one(a); put_line('a=', a); end test;
```



4.4 声明语句的翻译

④ 换名调用

严格讲，换名调用并不能算作真正的过程调用和参数传递。

换名调用由Algol的复写规则定义：

1. 过程被看作宏，每次对过程的调用，实质上是用**过程体替换过程调用**，**替换中用实参的文字替换体中的形参**。这样的替换方式被称为**宏替换**或**宏展开**；
2. 区分被调用过程的局部名和调用过程的名字。宏展开前被调用过程的每个局部名被**重新命名**成可区别的名字；

换名调用的C形式是**宏定义**(#define)，在预处理时进行宏替换，将过程体直接展开在它被调用的地方，消除了过程调用和参数传递，运行速度快。



4.4 声明语句的翻译

正文替换与函数调用的不一致性

//换名调用副作用的演示程序

```
#include <iostream.h>
```

```
int temp;
```

```
#define swap(x, y) temp=x; x=y; y=temp; // 宏定义
```

```
void swap(int &x, int &y) // 引用调用
```

```
{ int temp; temp=x; x=y; y=temp; }
```

```
void main ()
```

```
{ int a(1), b[]={0,0};
```

```
cout<<"before: a="<<a<<" b="<<b[0]<<" "<<b[1]<<endl;
```

```
swap(a, b[a]);
```

```
cout<<"after: a="<<a<<" b="<<b[0]<<" "<<b[1]<<endl;
```

```
}
```



4.4 声明语句的翻译

3. 作用域信息的保存

① 过程的作用域

与程序块类似，在允许**嵌套定义过程**的程序设计语言中，**相同的名字可以同时出现在不同的作用域中**，因此有必要讨论如何设计符号表来存放它们。此处讨论的过程作用域，同样遵守**静态作用域**和**最近嵌套原则**。

定义4.2 设主程序(最外层过程)的嵌套深度 $d_{\text{main}}=1$ ，则

- ① 若过程A直接嵌套定义过程B，则 $d_B = d_A + 1$ ；
- ② 变量声明时所在过程的嵌套深度，被认为是该**变量的嵌套深度**。

与程序块相比，有两点不同，使得过程声明的处理复杂：

- ① 过程头(即界面)是一个名字，可像引用变量一样被调用
- ② 程序块的执行(控制流)与静态一致，而过程不一致。



例4.14 快排序的Pascal程序：

```
program sort(input,output);
```

```
  var a:array[0..10]of integer; x:integer;
```

```
  procedure readarray;
```

```
  var i:integer;
```

```
  begin for i:=1 to 9 do read(a[i]) end{readarray};
```

```
  procedure exchange(i,j:integer);
```

```
  begin x:=a[i]; a[i]:=a[j]; a[j]:=x; end{exchange};
```

```
  procedure quicksort (m,n:integer );
```

```
  var i,v:integer;
```

```
    function partition(y,z:integer):integer;
```

```
    var i,j:integer;
```

```
    begin ...a...; ...v...; ...exchange(i,j);... end{partition};
```

```
  begin if (n>m) then
```

```
    begin i:=partition(m,n); quicksort(m,i-1); quicksort(i+1,n) end;
```

```
  end{quicksort};
```

```
begin a[0]:=-9999; a[10]:=9999; readarray; quicksort(1,9) end{sort}.
```

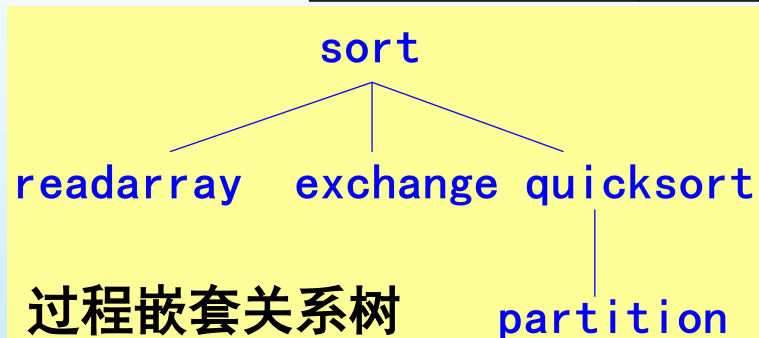
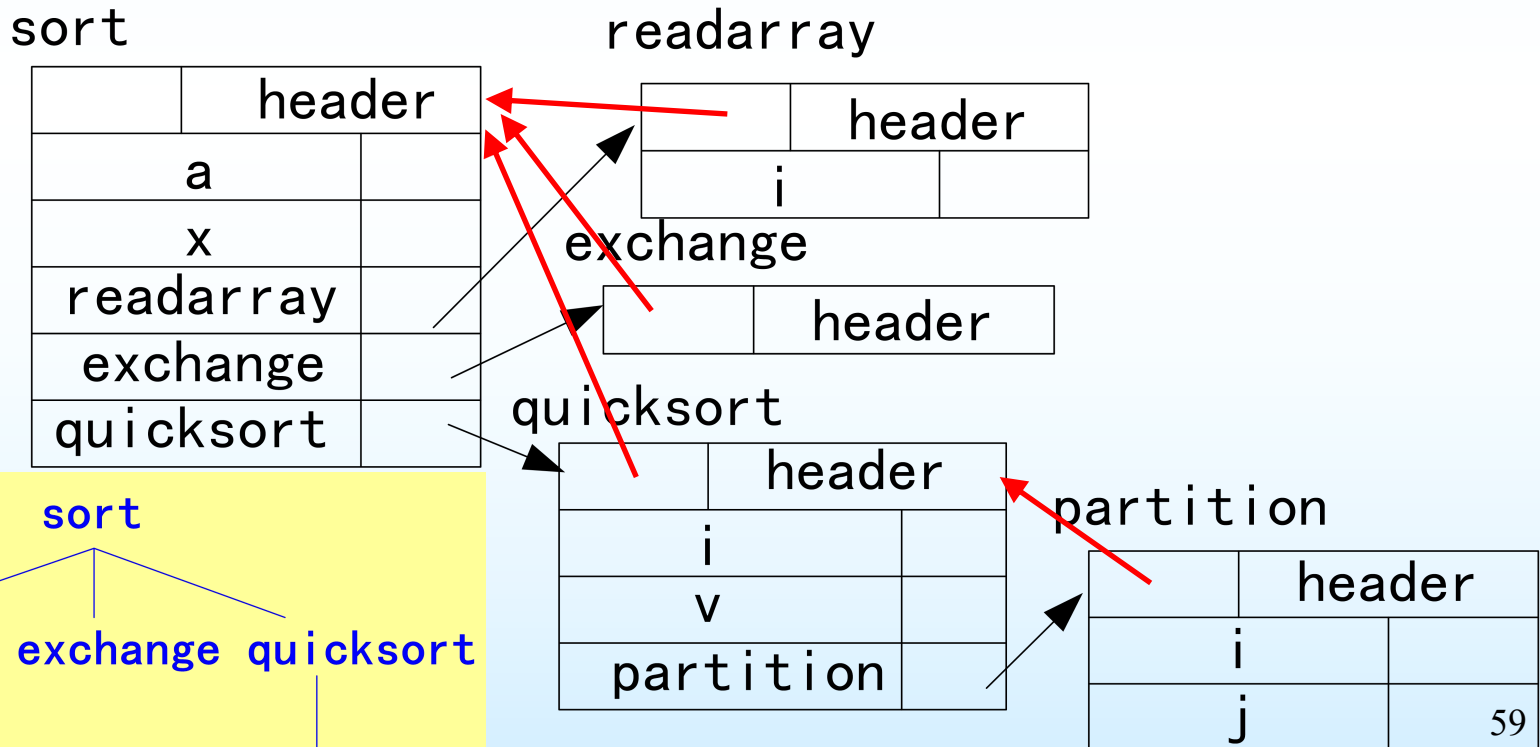
过程	变量	深度
sort	a, x	1
readarray	i	2
exchange		2
quicksort	i, v	2
partition	i, j	3



4.4 声明语句的翻译

② 符号表中的作用域信息

嵌套过程中的名字作用域信息，使用有嵌套结构的符号表保存。每个过程被认为是一个子符号表，或者是符号表中的一个节点。嵌套的节点之间用**双向链连接**，**正向链指示过程的嵌套关系**，**逆向链实现按作用域对名字进行访问**。



4.4 声明语句的翻译

③ 语法制导翻译生成符号表

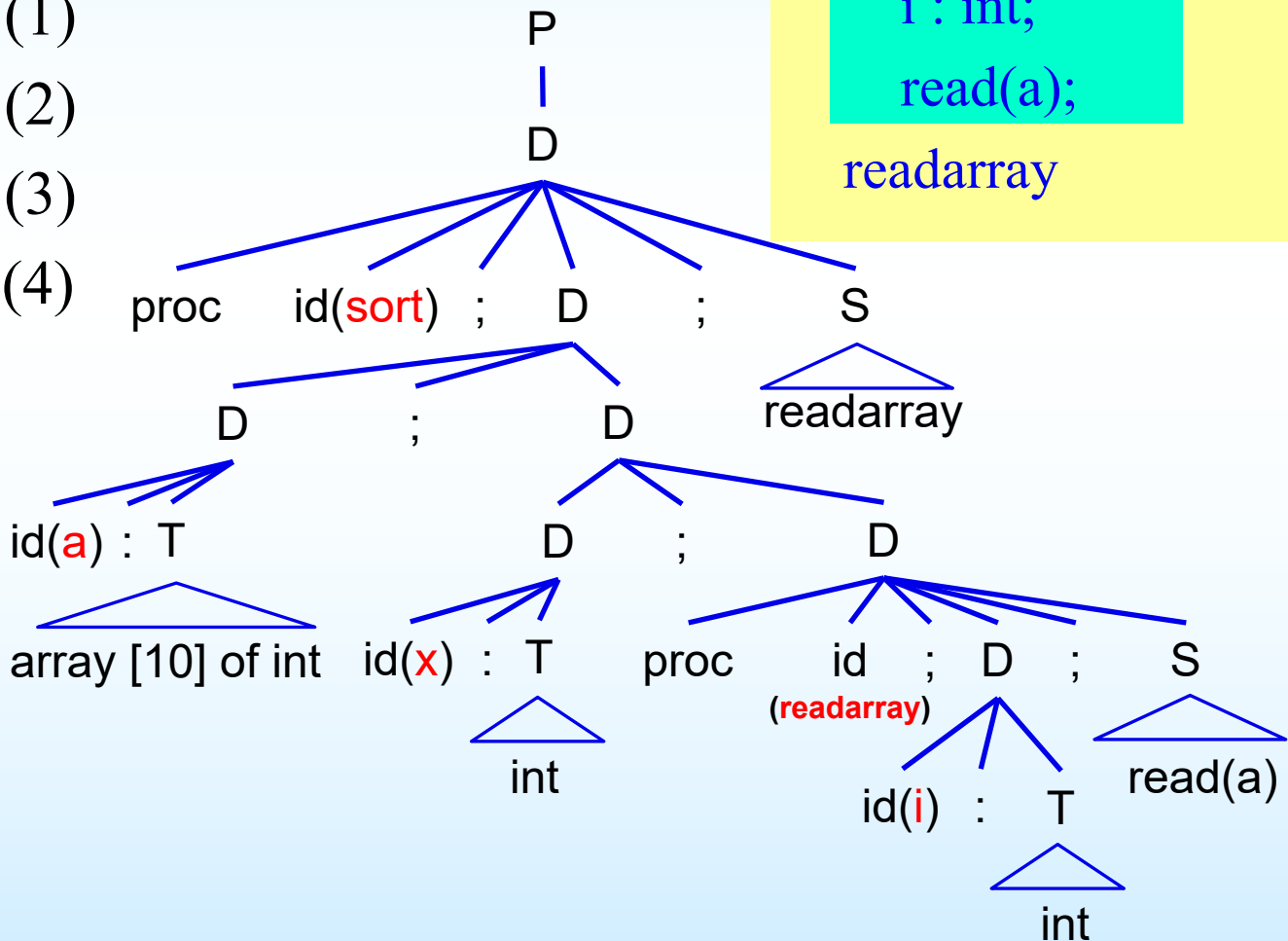
a) 简化的过程定义文法(忽略了参数)

$P \rightarrow D$ (1)

$D \rightarrow D ; D$ (2)

$| id : T$ (3)

$| proc\ id ; D ; S$ (4)



生成主程序符号表节点

生成sort

符号表节点

生成readarray

符号表节点

proc sort;

a : array[10] of int;

x : int;

proc readarray;

i : int;

read(a);

readarray

问题：如何在处理产生式(1)和(4)的语言结构时正确地填写符号表信息(双向链表)?



4.4 声明语句的翻译

③ 语法制导翻译生成符号表

a) 简化的过程定义文法(忽略了参数)

$P \rightarrow D$ (1)

$D \rightarrow D ; D$ (2)

| $id : T$ (3)

| $proc\ id ; D ; S$ (4)

问题：如何在处理产生式(1)和(4)的语言结构时正确地填写符号表信息(双向链表)?

修改文法，在定义D之前生成符号表(LR分析)

$P \rightarrow M D$ (1)

$D \rightarrow D ; D$ (2)

| $id : T$ (3)

| $proc\ id ; N D ; S$ (4)

$M \rightarrow \varepsilon$ (5) {生成主程序对应的符号表节点}

$N \rightarrow \varepsilon$ (6) {生成嵌套定义过程的符号表节点}⁶¹



4.4 声明语句的翻译

b) 全程量、属性与语义函数

1. 全程量：有序对栈(tblptr, offset)

(符号表节点的指针, 符号表节点所需宽度)

2. 栈上的操作：push(t, o)、pop、top(stack)

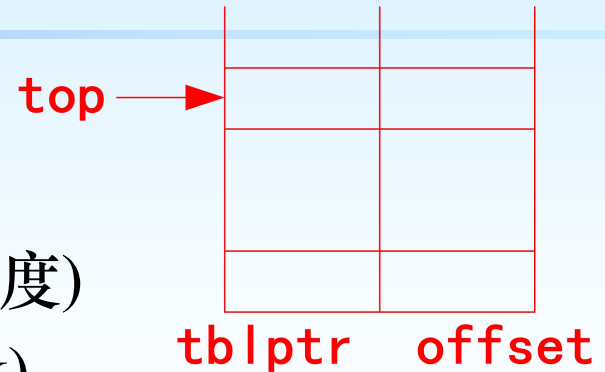
3. 语义函数与过程：

① **函数**mktable(previous)：建立一个新的节点，返回指向新节点的指针。**previous**是**逆向链**，指向该节点的前驱(外层)。

② **过程**enter(table, name, type, offset)：在table指向的节点中为名字name建立新的条目，包括名字的类型和存储位置等。

③ **过程**addwidth(table, width)：将width统计的table节点中所有条目的累加宽度，记录在table的头部信息中。

④ **过程**enterproc(table, name, newtable)：为**过程**name在table指向的节点中建立一个新的条目。**newtable**是**正向链**，指向name过程自身的符号表节点。





4.4 声明语句的翻译

c) 语义规则

- (1) $P \rightarrow \mathbf{M} D$ $\{\text{addwidth}(\text{top}(\text{tblptr}), \text{top}(\text{offset})); \text{pop};\}$
- (2) $\mathbf{M} \rightarrow \varepsilon$ $\{t := \text{mktable}(\text{null}); \text{push}(t, 0);\}$
- (3) $D \rightarrow D ; D$
- (4) $D \rightarrow \text{id} : T$ $\{\text{enter}(\text{top}(\text{tblptr}), \text{id.name}, T.\text{type}, \text{top}(\text{offset}));$
 $\text{top}(\text{offset}) := \text{top}(\text{offset}) + T.\text{width};\}$
- (5) $D \rightarrow \text{proc id} ; \mathbf{N} D1 ; S$ $\{t := \text{top}(\text{tblptr});$
 $\text{addwidth}(t, \text{top}(\text{offset}));$
 $\text{pop};$
 $\text{enterproc}(\text{top}(\text{tblptr}), \text{id.name}, t);\}$
- (6) $\mathbf{N} \rightarrow \varepsilon$ $\{t := \text{mktable}(\text{top}(\text{tblptr})); \text{push}(t, 0);\}$

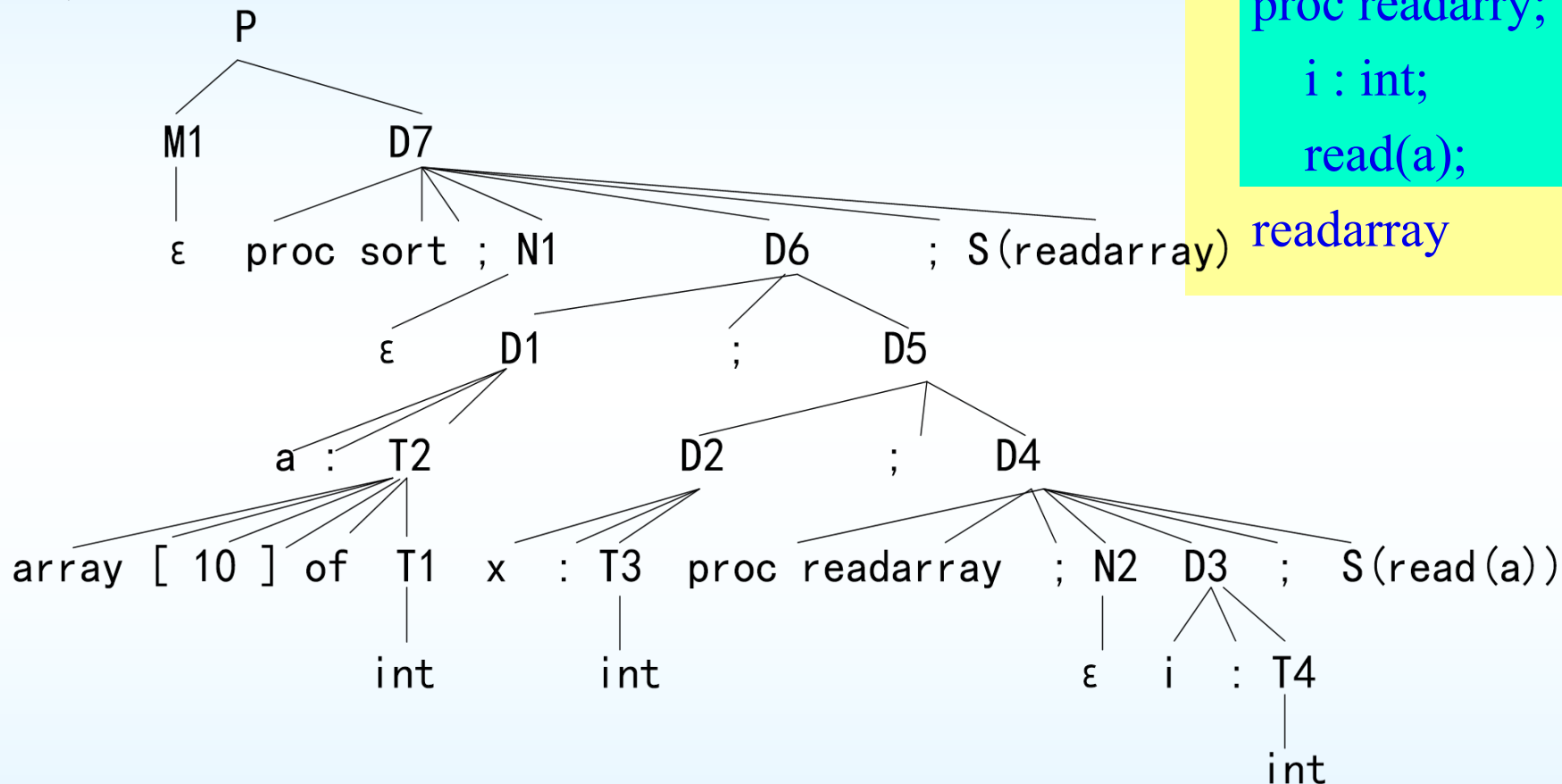
```
proc sort;  
  a : array[10] of int;  
  x : int;
```

```
i : int;
```

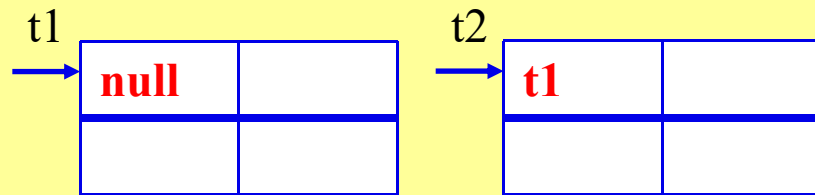
```
read(a);
```

readarray

d) 语法制导翻译的过程

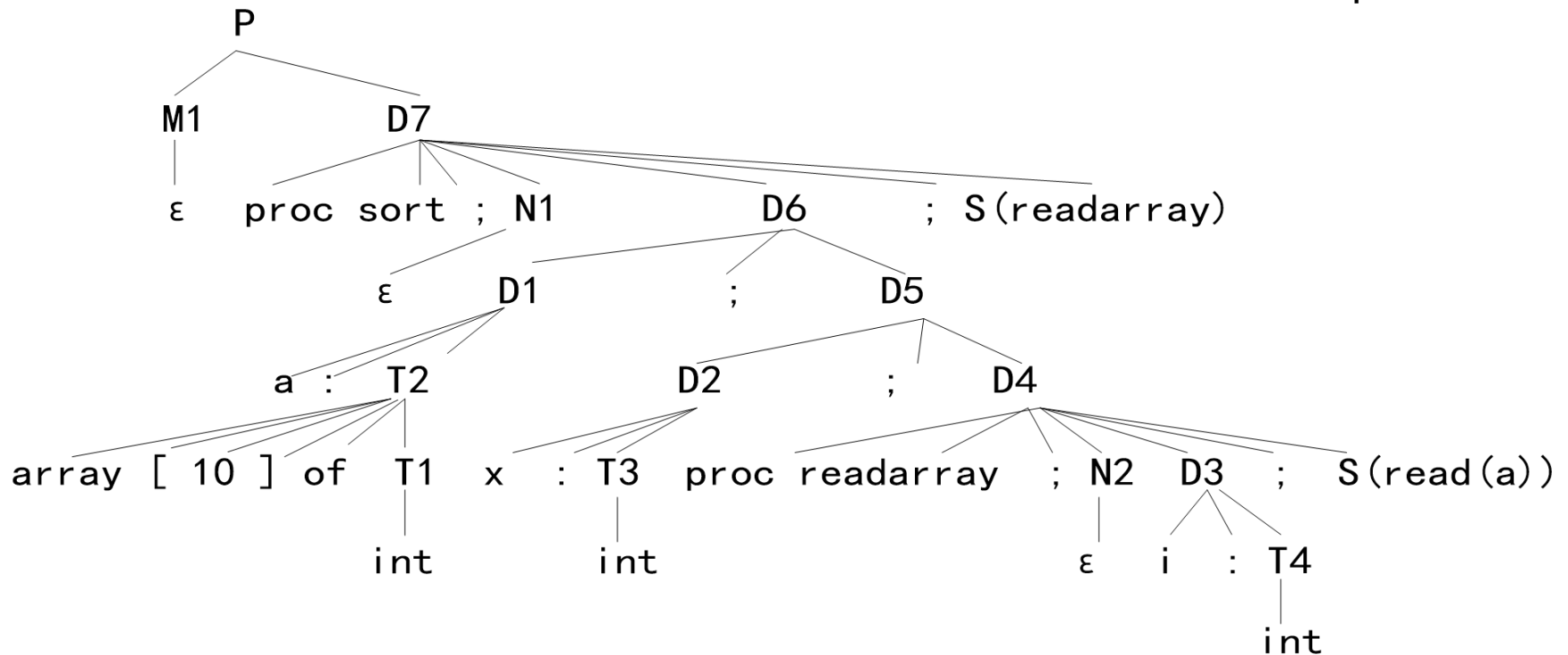


- (1) $M1 \rightarrow \varepsilon$ $t1 := \text{mktable}(\text{null}); \text{push}(t1, 0);$
- (2) $N1 \rightarrow \varepsilon$ $t2 := \text{mktable}(\text{top}(\text{tblptr})); \text{push}(t2, 0);$
- (3) $T1 \rightarrow \text{int}$ $T1.\text{type} = \text{integer}, T1.\text{width} = 4$
- (4) $T2 \rightarrow \text{array } [10] \text{ of } T1$ $T2.\text{type} = \text{array}(10, \text{int}), T2.\text{width} = 40$



t2	0
t1	0

tblptr offset

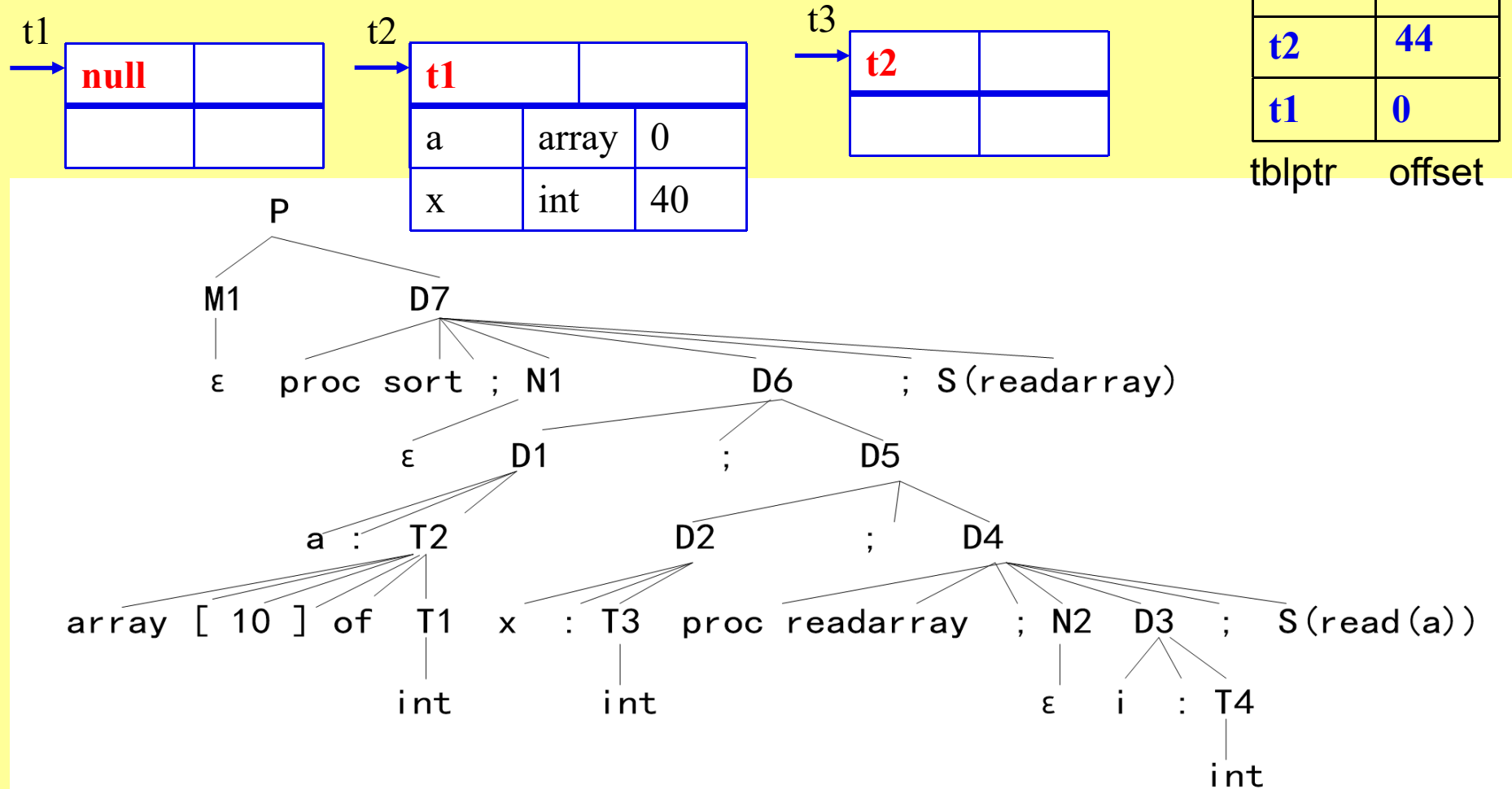


(5) $D1 \rightarrow a:T2$ (a,arr,0)填进t2所指节点, $top(offset) := 40$

(6) $T3 \rightarrow int$ $T2.type = integer$, $T2.width = 4$

(7) $D2 \rightarrow x:T3$ (x,int,40)填进t2所指节点, $top(offset) := 44$

(8) $N2 \rightarrow \epsilon$ $t3 := mktable(top(tblptr)); push(t3, 0);$



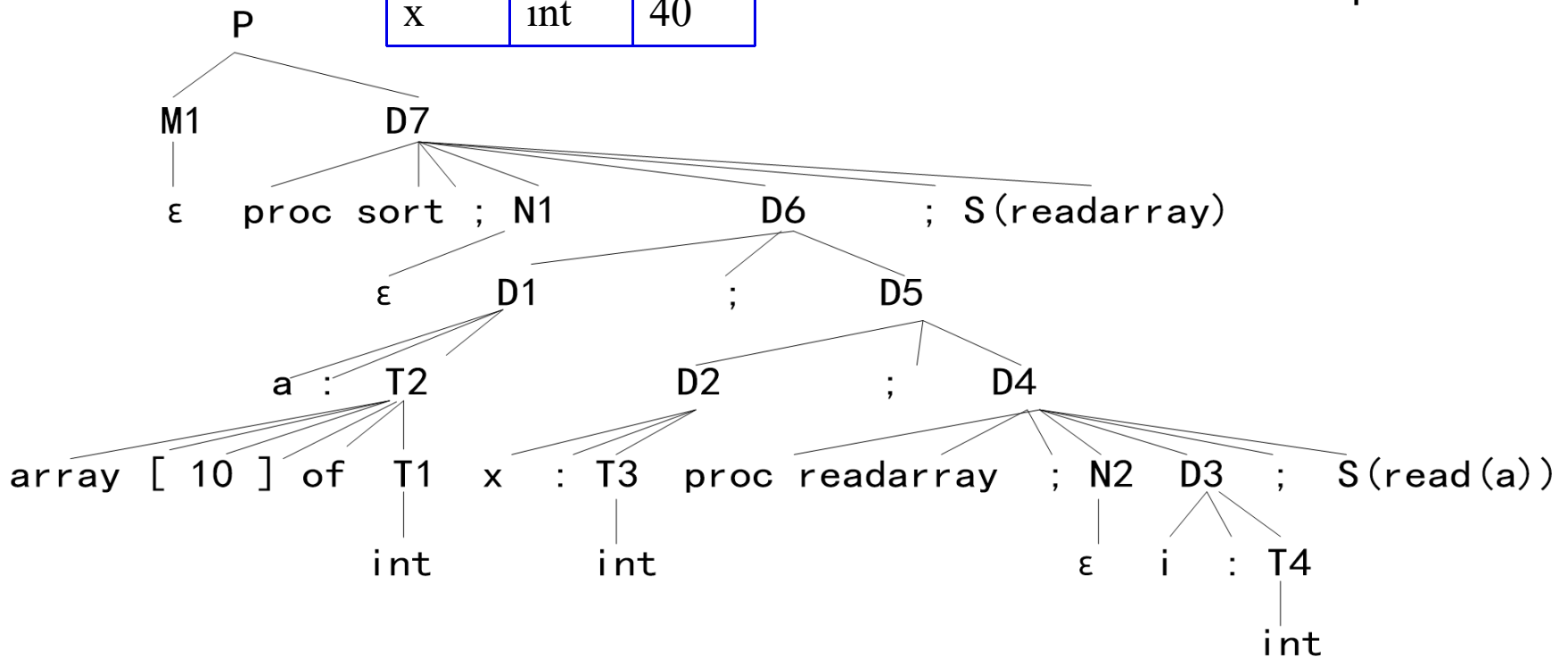
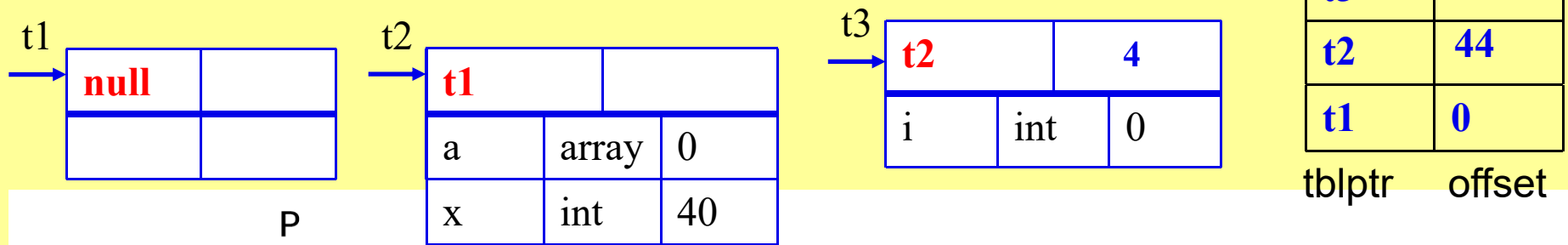
(9) $T4 \rightarrow \text{int}$ $T4.\text{type} = \text{integer}$, $T4.\text{width} = 4$

(10) $D3 \rightarrow i:T4$ ($i, \text{int}, 0$) 填进 $t3$ 所指节点, $\text{top}(\text{offset}) := 4$

(11) $D4 \rightarrow \text{proc readarray } N2 \ D3 ; S$

$t := \text{top}(\text{tblptr}); \text{addwidth}(t, \text{top}(\text{offset}));$

$\text{pop}; \text{enterproc}(\text{top}(\text{tblptr}), \text{readarray}, t);$



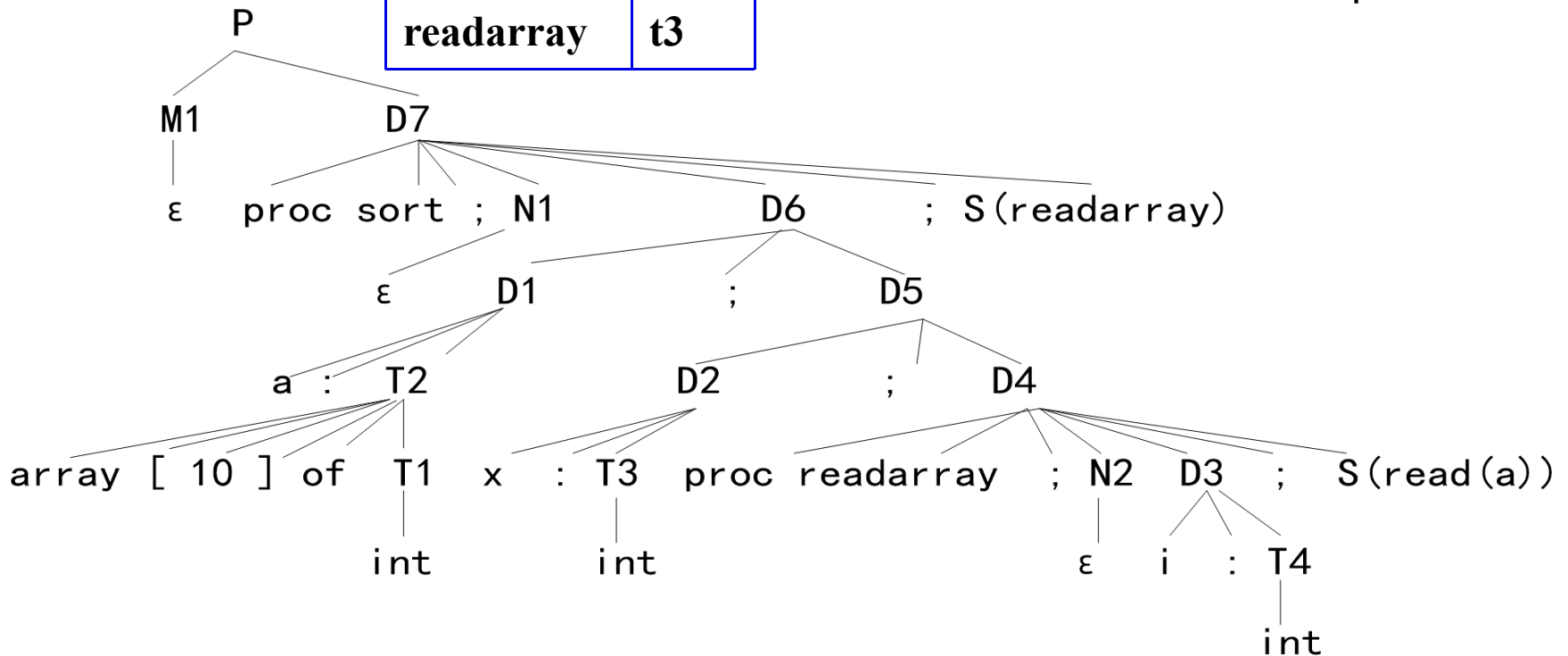
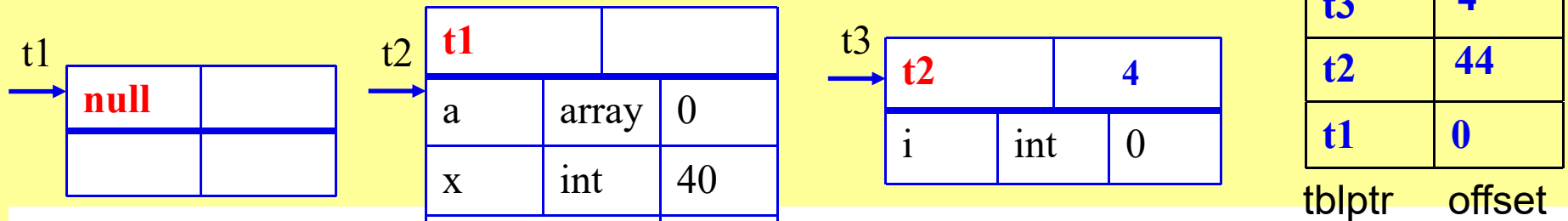
(9) $T4 \rightarrow \text{int}$ $T4.\text{type} = \text{integer}$, $T4.\text{width} = 4$

(10) $D3 \rightarrow i:T4$ (i,int,0)填进t3所指节点, $\text{top}(\text{offset}) := 4$

(11) $D4 \rightarrow \text{proc readarray } N2 \ D3 ; S$

$t := \text{top}(\text{tblptr}); \text{addwidth}(t, \text{top}(\text{offset}));$

$\text{pop}; \text{enterproc}(\text{top}(\text{tblptr}), \text{readarray}, t);$



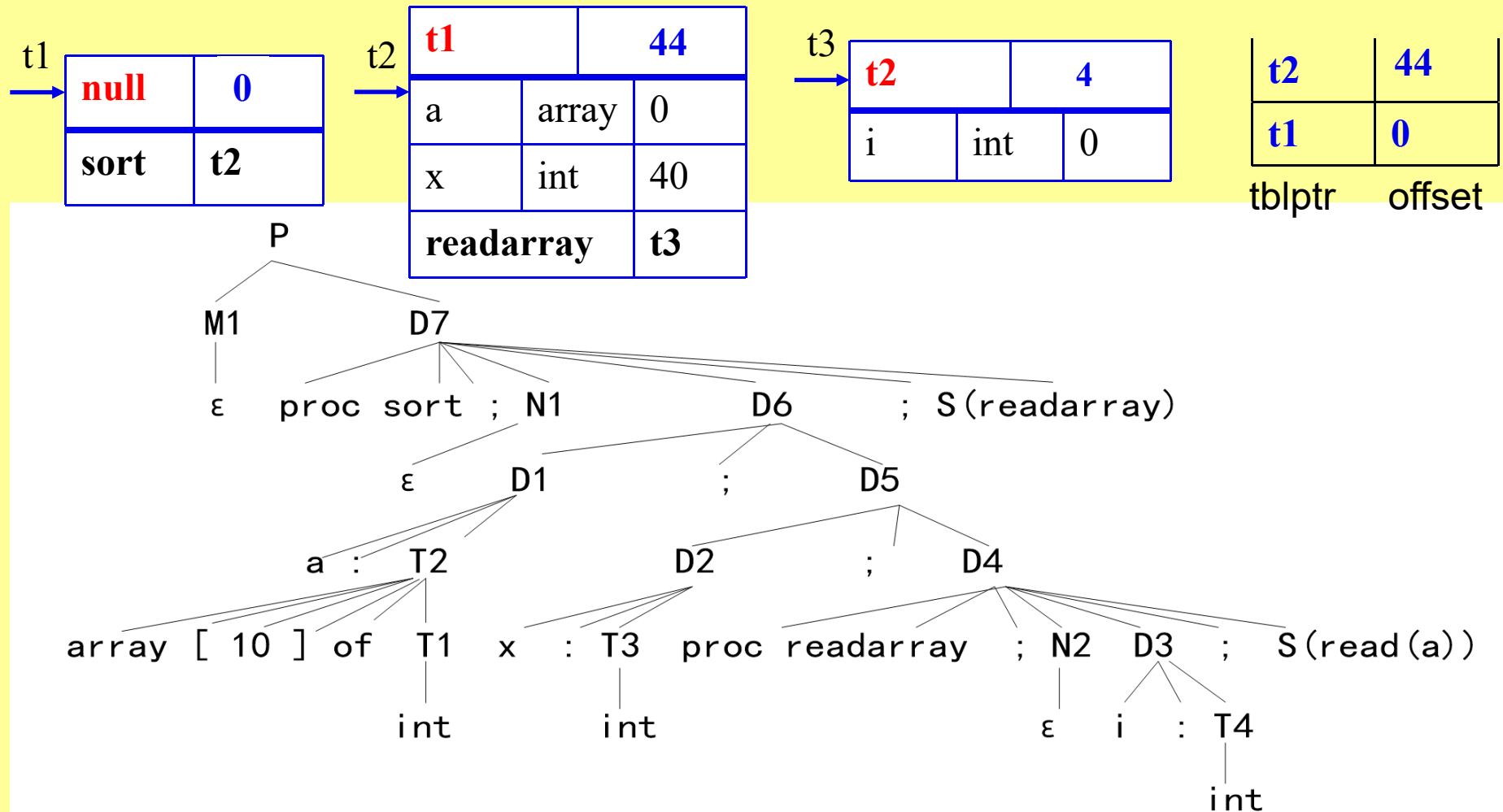
(14) D7 → proc sort N1 D6 ; S

t := top(tblptr); addwidth(t, top(offset));

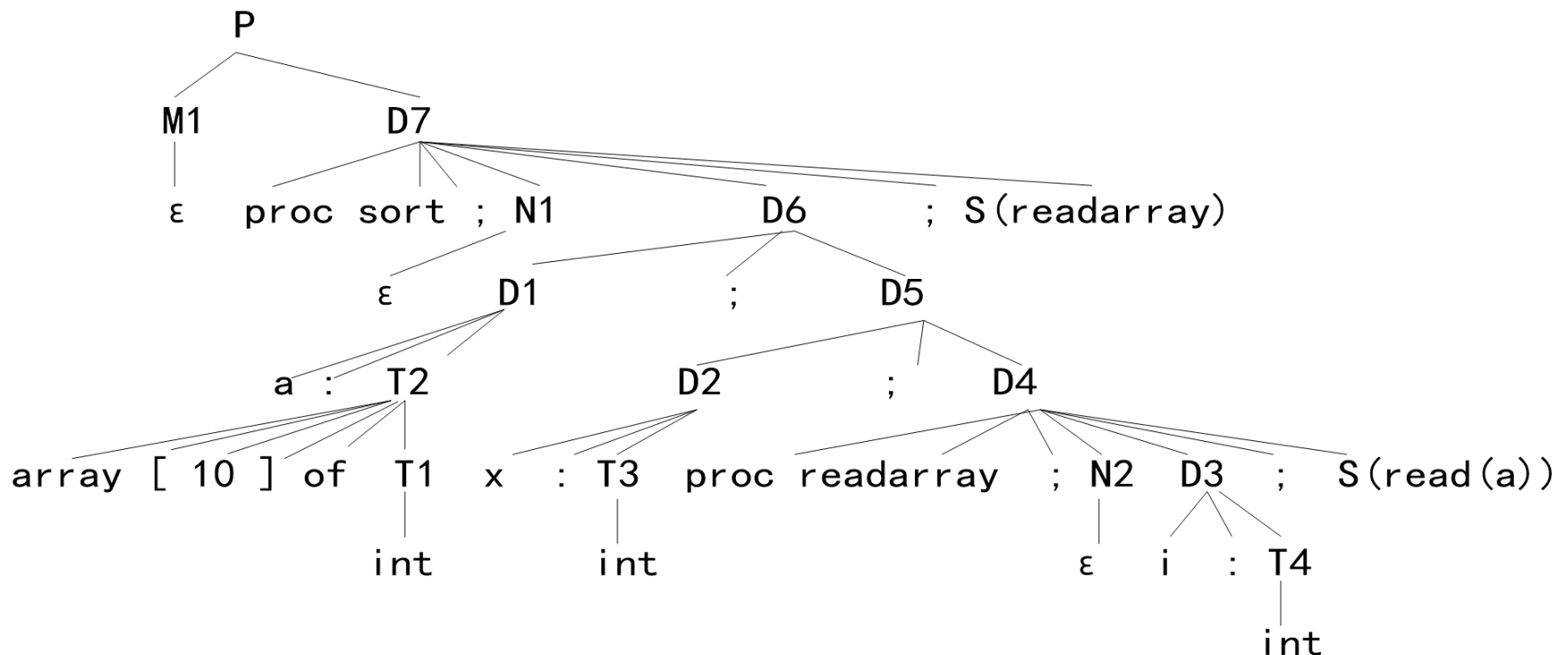
pop; enterproc(top(tblptr), sort, t);

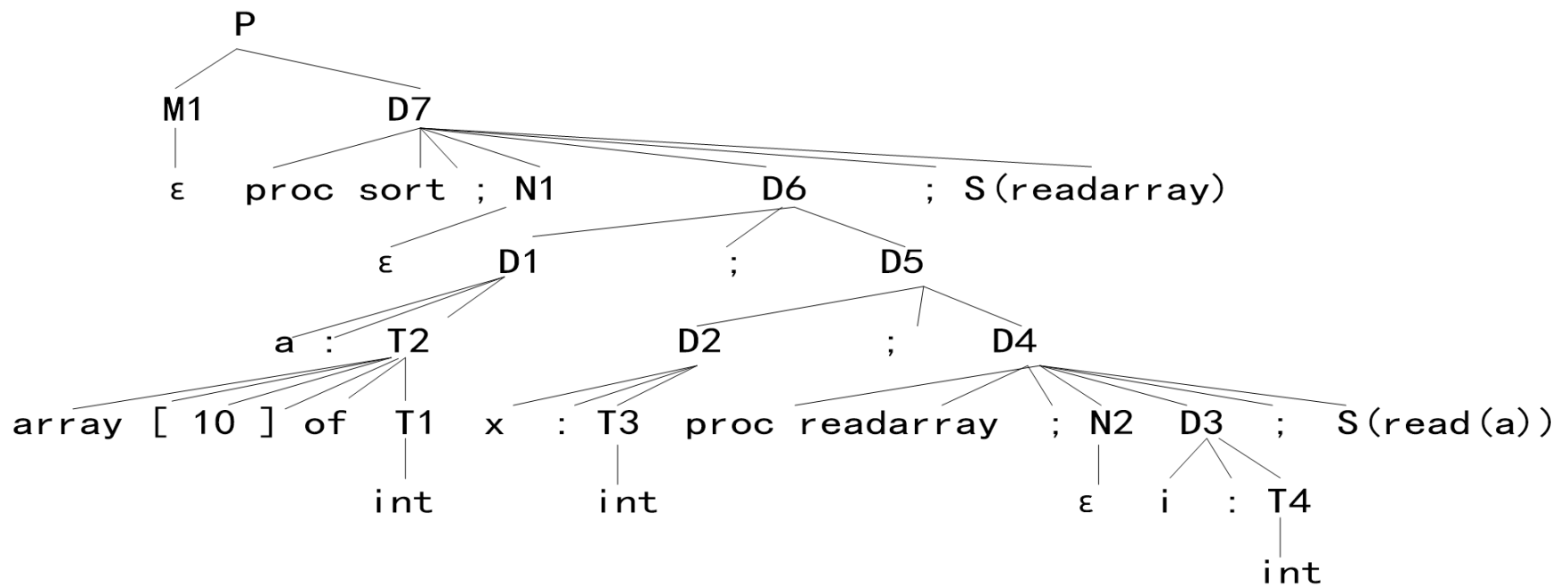
(15) P → M1 D7

addwidth(top(tblptr), top(offset)); pop;



- (1) $M1 \rightarrow \varepsilon$ $t1 := \text{mktable}(\text{null}); \text{push}(t1, 0);$
- (2) $N1 \rightarrow \varepsilon$ $t2 := \text{mktable}(\text{top}(\text{tblptr})); \text{push}(t2, 0);$
- (3) $T1 \rightarrow \text{int}$ $T1.\text{type} = \text{integer}, T1.\text{width} = 4$
- (4) $T2 \rightarrow \text{array } [10] \text{ of } T1$ $T2.\text{type} = \text{array}(10, \text{int}), T2.\text{width} = 40$
- (5) $D1 \rightarrow a:T2$ $(a, \text{arr}, 0)$ 填进 $t2$ 所指节点, $\text{top}(\text{offset}) := 40$
- (6) $T3 \rightarrow \text{int}$ $T2.\text{type} = \text{integer}, T2.\text{width} = 4$
- (7) $D2 \rightarrow x:T3$ $(x, \text{int}, 40)$ 填进 $t2$ 所指节点, $\text{top}(\text{offset}) := 44$
- (8) $N2 \rightarrow \varepsilon$ $t3 := \text{mktable}(\text{top}(\text{tblptr})); \text{push}(t3, 0);$





(9) $T4 \rightarrow \text{int}$ $T4.\text{type}=\text{integer}$, $T4.\text{width}=4$

(10) $D3 \rightarrow i:T4$ ($i, \text{int}, 0$)填进t3所指节点, $\text{top}(\text{offset}) := 4$

(11) $D4 \rightarrow \text{proc readarray } N2 \ D3 ; S$

$t := \text{top}(\text{tblptr}); \text{addwidth}(t, \text{top}(\text{offset}));$

$\text{pop}; \text{enterproc}(\text{top}(\text{tblptr}), \text{readarray}, t);$

(14) $D7 \rightarrow \text{proc sort } N1 \ D6 ; S$

$t := \text{top}(\text{tblptr}); \text{addwidth}(t, \text{top}(\text{offset}));$

$\text{pop}; \text{enterproc}(\text{top}(\text{tblptr}), \text{sort}, t);$

(15) $P \rightarrow M1 \ D7$ $\text{addwidth}(\text{top}(\text{tblptr}), \text{top}(\text{offset})); \text{pop};$